

DKEX

(Ding Key Exchange)

June 30, 2026

Submitted To
Institute of Commercial Cryptography Standards

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Contributions	4
1.3	Design Objectives	5
2	Algorithm Description	6
2.1	Preliminaries	6
2.1.1	Ring, Modules, and Notation	6
2.1.2	Noise Distributions	8
2.1.3	Public Parameters	9
2.1.4	Matrix Generation and Sampling Algorithms	10
2.2	Core Functions	11
2.2.1	Reconciliation Mechanism	11
2.2.2	Number-Theoretic Transform (NTT)	13
2.2.3	Compression and Decompression Functions	15
2.3	Specification of DKE	15
2.4	Specification of DKEX	16
2.5	Parameter Sets	21
3	Design Rationale	21
3.1	The mathematical hard problem	21
3.2	Criteria for Parameter Selection	21
3.3	Failure probability analysis.	22
3.4	Other considerations	22
4	Performance Evaluation	23
5	Security Statements and Analyses	25
5.1	Theoretical security	25
5.1.1	Security Model	25
5.1.2	CK security with wPFS	28
5.2	Analysis with respect to known attacks	31
5.2.1	Attacks against the underlying MLWE problem	31
5.2.2	Refined gate-count estimate	33
5.2.3	About quantum security	35
5.2.4	Attacks against symmetric primitives	35
5.2.5	Attacks exploiting reconciliation failures	36
5.3	Other security properties	36
6	Feature Statements	37

1 Introduction

1.1 Background and Motivation

The progress of quantum computing has changed the security landscape for public-key cryptography. Public-key cryptosystems based on integer factorization and discrete logarithms, such as RSA and classical Diffie-Hellman key exchange, can be broken in polynomial time by a large-scale quantum computer using Shor’s algorithm [1]. This observation has motivated the development of post-quantum cryptographic primitives whose security is expected to remain valid against quantum adversaries.

In the evolving landscape of post-quantum cryptography, lattice-based cryptography stands out as one of the most mature and efficient practical approaches available. The Learning with Errors (LWE) problem and its structured variants [2, 3, 4] provide the foundation for many encryption, key-establishment, and signature schemes. In key establishment, lattice-based methods differ fundamentally from classical Diffie-Hellman techniques. Instead of deriving identical algebraic values, the two parties generally compute related, noisy values. They then utilize a reconciliation mechanism to convert these similar values into a common bit string.

Ding Key Exchange is an early representative of this reconciliation-based approach [5, 6]. The protocol works in an LWE-type setting and shows how two parties can derive a common key from noisy algebraic samples. The exchanged values do not directly produce identical secrets; rather, they produce values that are close enough for a reconciliation procedure to recover matching key bits with high probability. This idea is important because it shows how noise, which is essential for LWE security, can still be made compatible with reliable key agreement.

However, the original Ding-style setting is not directly aligned with the module-lattice ecosystem that has become dominant in recent post-quantum standardization. Modern lattice-based standards use structured modules over polynomial rings to obtain compact public data, efficient arithmetic, and practical implementations. In particular, the scheme formerly known as Kyber [7] has been standardized by NIST as ML-KEM in FIPS 203 [8], while the scheme formerly known as Dilithium has been standardized as ML-DSA in FIPS 204 [9]. These standards demonstrate that Module-LWE-based constructions can provide a strong balance between security, efficiency, and implementability.

This raises an important question: can the reconciliation-based intuition of the Ding Key Exchange be reformulated within a modern module-lattice framework and be used as the foundation for an authenticated key exchange protocol? Such a construction would be beneficial because it would provide an explicit interactive key-exchange mechanism that remains compatible with the algebraic and implementation choices of the ML-KEM and ML-DSA ecosystem.

Authenticated key exchange also requires protection against active adversaries. A passive key exchange only ensures that two honest parties can derive a shared secret when messages are delivered honestly. In a real network, an adversary may modify, inject, replay, or reorder messages. Therefore, the public key-exchange transcript must be authenticated and bound to the intended parties. Classical signature-authenticated protocols achieve this by signing the exchanged ephemeral values. In the post-quantum setting, the same principle can be applied using a post-quantum signature scheme.

The goal of this work is therefore to combine a module-lattice Ding-type key exchange with a signature-based authentication layer. The resulting protocol follows a modular design: the module-lattice key exchange produces a raw shared secret, the signature layer authenticates the public transcript, and a key derivation function binds the final session key to the transcript. This framework effectively supports a CK security model [10], enabling separate analysis of the authentication and key-indistinguishability components.

1.2 Contributions

This work builds on these developments by considering an improved Ding-type key exchange in the module-lattice setting. The underlying unauthenticated key exchange, denoted DKE, can be viewed as a module-lattice adaptation of Ding Key Exchange with a refined reconciliation mechanism. Instead of working over vector spaces such as $\mathbb{Z}_q^n$, the construction works over modules R_q^k . The public LWE matrix is replaced by a structured Module-LWE matrix $\mathbf{A} \in R_q^{k \times k}$, generated from a short public seed in the same spirit as ML-KEM. This allows the construction to inherit the compactness and efficient arithmetic of module-lattice designs.

A central part of the construction is the improved reconciliation mechanism. The parties compute noisy module values from exchanged public module elements, secret vectors, and small error terms. These noisy values are not exactly equal, but they are sufficiently close for reconciliation to recover the same bit string except with negligible probability. The reconciliation procedure is adapted to the module-lattice setting and tuned to the modulus, noise distribution, and correctness requirements of an ML-KEM-like parameter environment. In this way, the construction keeps the conceptual simplicity of Ding Key Exchange while making it more suitable for modern module-lattice implementations.

This work transforms the unauthenticated key exchange into an authenticated one, namely DKEX. The protocol uses a digital signature layer to authenticate public key exchange messages. The initiator sends a public message M_1 , the responder returns a public response M_2 together with a signature, and the initiator completes the protocol by returning its own signature. Both parties compute the common transcript hash

$$T \leftarrow H(\text{LABEL}_A, \text{LABEL}_B, pk_A, pk_B, M_1, M_2),$$

and derive the final session keys as

$$K_A = \text{KDF}(T, z_A), \quad K_B = \text{KDF}(T, z_B),$$

where z_A and z_B are the raw shared secrets obtained from DKE. If the reconciliation succeeds, then $z_A = z_B$, and both parties derive the same final key $K_A = K_B$.

The transcript hash plays two roles. First, it provides a compact value that both parties sign and verify, thereby authenticating the public keys and the exchanged messages used as public context in the final key derivation, so that the session key is bound to the ordered signing keys and the exchanged values M_1 and M_2 . This transcript binding is included to prevent identity-misbinding phenomena such as unknown-key-share and bilateral unknown-key-share patterns [11, 12].

The security analysis is given in a CK model with weak Perfect Forward Secrecy (wPFS) [10, 13]. The proof follows the structure of classical signature-authenticated key exchange. If an honest party accepts without a matching honest peer session, then the adversary must either forge a signature or find a collision in the transcript hash, apart from negligible session-identifier collision events. If a matching session exists, then the secrecy of the final key is reduced to the derived-key indistinguishability of the DKE and KDF pair.

The resulting construction is intended to be both theoretically modular and practically compatible with module-lattice cryptography. The design separates the key-exchange component, the signature component, the transcript hash, and the KDF. This modularity allows each component to be specified, analyzed, and potentially replaced independently.

1.3 Design Objectives

Security. The main goal of this proposal is to obtain an Authenticated Key Exchange (AKE) protocol whose security can be reduced to well-defined post-quantum assumptions. The secrecy of the session key is based on the hardness of Module-LWE over the ring R_q . Authentication is based on the signature scheme’s unforgeability. The target AKE model is a CK model with weak perfect forward secrecy.

Negligible failure probability. Since DKE relies on a reconciliation-based approach, its correctness is not entirely deterministic. Thus, the protocol must ensure that the failure probability is minimal for the chosen parameter set. The reconciliation mechanism has been revised to ensure that the failure probability is consistent with current lattice-based schemes and is much lower than the original version. This architectural enhancement effectively eliminates the higher failure rates observed in early Ding Key Exchange variants (e.g. 2^{-60}), reducing the decryption failure probability to cryptographically negligible levels (e.g. $\leq 2^{-132.7}$).

Robust algebraic foundations and ecosystem compatibility. The construction is based on module-lattice arithmetic over R_q^k , with public matrices generated from short public seeds. This follows the design philosophy of ML-KEM and supports compact public parameters, efficient arithmetic, and compatibility with existing module-lattice implementation techniques. The Module-LWE structure offers a practical balance between performance and conservative algebraic modeling, making it easier to compare with standardized post-quantum primitives.

Efficiency and implementability. The protocol is intended to be efficient in both communication and computation. Public matrix generation from a seed reduces bandwidth and storage. Module arithmetic can benefit from optimized polynomial multiplication and number-theoretic-transform techniques already developed for ML-KEM-like schemes. The AKE layer adds transcript hashing, digital signatures, and KDF computation. This keeps the construction implementable in software and potentially suitable for hardware or constrained-device optimization.

Simplicity and modularity. The construction separates the unauthenticated key exchange from the authentication layer. The key exchange component produces raw shared secrets; the signature component authenticates the public transcript; and the KDF derives the final session key from the raw secret and transcript. This modularity greatly streamlines the security proof, making it both clearer and more robust.

Transcript binding and resistance to identity misbinding. The final key is derived from a transcript hash containing the ordered signing public keys and the public module-lattice messages. This prevents two different perceived sessions from deriving the same final key merely because they share the same raw key-exchange value. The transcript-bound KDF provides channel binding and resistance to unknown-key-share and bilateral unknown-key-share patterns. This feature is included to enhance security and reduce implementation errors.

Suitability for standardization-oriented evaluation. A proposal focused on standardization should be straightforward to specify, implement, test, and compare. The suggested authenticated key exchange utilizes a limited number of components: a key exchange based on module-lattice reconciliation, a transcript hash, a key derivation function (KDF), and a signature scheme. The message formats can be encoded in a standard way, the

correctness error can be clearly defined, and the proof distinctly separates the assumptions made about each component.

2 Algorithm Description

2.1 Preliminaries

In this subsection we describe the algebraic setting, define various notations and the noise distributions used throughout the scheme, fix the public parameters and notation, and describe the generation of the public matrix $\mathbf{A}$ from a seed.

2.1.1 Ring, Modules, and Notation

Base field and modulus. Let q be an odd prime. We write $\mathbb{Z}_q$ for the finite field with q elements, and we identify it with the set of integers $\{-\frac{q-1}{2}, \dots, 0, \dots, \frac{q-1}{2}\}$ when discussing magnitudes or norms. All additions and multiplications of coefficients are taken modulo q unless otherwise stated. In this specification, $q = 3329$ for the two firsts levels of security, and is incremented to $q = 7681$ for the last security level.

Polynomial ring. Consider the polynomial ring

$$R_q = \mathbb{Z}_q[X]/(X^n + 1),$$

where $n = 256$ for the first two security levels and $n = 512$ for the third. An element $a \in R_q$ is represented by a polynomial

$$a(X) = a_0 + a_1X + \dots + a_{n-1}X^{n-1} \quad \text{with } a_i \in \mathbb{Z}_q,$$

where polynomials are reduced modulo $X^n + 1$, and coefficient-wise modulo q . We will often identify a with the coefficient vector $(a_0, \dots, a_{n-1}) \in \mathbb{Z}_q^n$.

Modules and matrices over R_q . For $k \geq 1$, we write R_q^k for the set of column vectors

$$\mathbf{v} = (v_1, \dots, v_k)^\top \quad \text{with } v_i \in R_q.$$

Matrices over R_q are denoted by bold uppercase letters: $\mathbf{A} \in R_q^{m \times k}$ is an $m \times k$ matrix whose entries are in R_q . Matrix-vector multiplication ($\mathbf{A}\mathbf{s} \in R_q^m$) and vector addition are defined in the natural way using ring operations in R_q .

Elements of R_q^k and $R_q^{m \times k}$ can be equivalently viewed as arrays of polynomials or as arrays of length- n coefficient vectors, depending on the context.

Inner products and norms. For $\mathbf{u}, \mathbf{v} \in R_q^k$, we define their R_q -valued inner product as

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^k u_i \cdot v_i \in R_q.$$

We also denote $\mathbf{u} \cdot \mathbf{v} = \langle \mathbf{u}, \mathbf{v} \rangle$.

When needed, we endow R_q and R_q^k with coefficient-wise norms. For $a \in R_q$ with coefficient vector $(a_0, \dots, a_{n-1})$ — lifted to integers in the canonical interval —, we define

$$\|a\|_\infty := \max_{0 \leq i < n} |a_i|, \quad \|a\|_2 := \left(\sum_{i=0}^{n-1} a_i^2 \right)^{1/2}.$$

These norms extend naturally to vectors $\mathbf{v} \in R_q^k$ by viewing $\mathbf{v}$ as a length- kn integer vector.

Number Theoretic Transform (NTT). Efficient polynomial multiplication in R_q is achieved using the Number Theoretic Transform (NTT) and its inverse, as in [8]. Recall that $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, with n a power of two and q an odd prime. The arithmetic depends on the factorization of $X^n + 1$. At first it was thought that full splitting was needed, that is, $2n \mid (q-1)$, but it was proved that having quadratic splitting is as efficient and this significantly reduces the needed modulus [14]. We are on the quadratic splitting case in all our parameter sets.

Suppose that $n \mid (q-1)$ but $2n \nmid (q-1)$. In this case, $\mathbb{Z}_q$ contains a primitive n -th root of unity but not a primitive $2n$ -th root. Choose $\zeta \in \mathbb{Z}_q$ such that $\zeta^n = 1$, and let $r := \log_2(n/2)$.

We define the r -bit reversal map: bit_rev_r , for $n = 2^r$, which maps an unsigned r -bit value $m = \sum_{s=0}^{r-1} a_s 2^s$ to an integer as:

$$\text{bit_rev}_r(m) = \sum_{s=0}^{r-1} a_s 2^{r-1-s}, \quad a_s \in \{0, 1\}.$$

The following factorization can be expressed in bit reversal:

$$X^n + 1 = \prod_{i=0}^{n/2-1} (x^2 - \zeta^{2\text{bit_rev}_r(i)+1}).$$

For $(n, q) = (256, 3329)$, we have:

$$x^{256} + 1 = \prod_{i=0}^{127} (x^2 - \zeta^{2\text{bit_rev}_7(i)+1}).$$

In this case $\zeta = 7$ is taken, since it is the smallest of all possible values. Similarly, for $(512, 7681)$ we have:

$$x^{512} + 1 = \prod_{i=0}^{255} (x^2 - \zeta^{2\text{bit_rev}_8(i)+1}),$$

being $\zeta = 62$ the choice in this case.

Therefore, the ring $R_q := \mathbb{Z}_q[X]/(X^n + 1)$ is isomorphic to T_q , where

$$T_q = \bigoplus_{i=0}^{n/2-1} \mathbb{Z}_q[X]/(X^2 - \zeta^{2\text{bit_rev}_r(i)+1}),$$

and the NTT representation $\hat{f} \in T_q$ of $f \in R_q$ is represented as:

$$f \mapsto \hat{f} = (f \bmod (X^2 - \zeta^{2\text{bit_rev}_r(i)+1}))_{i=0}^{n/2-1}.$$

Moreover, each transform block consists of two coefficients as:

$$f \bmod (X^2 - \zeta^{2\text{bit_rev}_r(i)+1}) = \hat{f}[2i] + \hat{f}[2i+1]X.$$

In this case, the output is split into consecutive pairs, and each pair represents a polynomial of degree < 2 in a quotient ring $\mathbb{Z}_q[X]/(X^2 - \Gamma)$. Hence, the natural multiplication subroutine is a degree-1 multiplication modulo $X^2 - \Gamma$. Algebraically,

$$(a_0 + a_1X)(b_0 + b_1X) = a_0b_0 + (a_0b_1 + a_1b_0)X + a_1b_1X^2,$$

and then X^2 is replaced by Γ . This multiplication is given by the algorithm `base_mul` (Alg. 6).

The concrete algorithms we use for the computation of these transformations and operations are provided in 2.2.2

Random oracles and XOFs. Throughout the specification, we assume the availability of cryptographic hash functions and extendable-output functions (XOFs) provided by ICCS. These are used for:

- Generating the public matrix $\mathbf{A}$ from a short seed.
- Sampling secrets and errors via rejection or bit-slicing.
- Deriving session keys from shared bit strings.

We denote generic hash/XOF functions by H, G , etc.; their precise instantiation follows SM3, HMAC-SM3 and KDF-SM3. These hash functions are used in the reference implementation just for correctness verification, with no security guarantee.

2.1.2 Noise Distributions

The security and correctness of DKE relies on carefully chosen distributions for secrets and error terms, mirroring ML-KEM. At a high level, these are *small* integer-valued distributions with support concentrated around 0 and short tails.

Centered binomial distribution. Let $\eta \in \mathbb{N}$ be a small integer parameter. The centered binomial distribution CBD_η over $\mathbb{Z}$ is defined as the distribution of the random variable

$$X = \sum_{i=1}^{\eta} (b_i - b'_i),$$

where $(b_i)_{i=1}^{\eta}$ and $(b'_i)_{i=1}^{\eta}$ are independent Bernoulli random variables with parameter $1/2$. Thus, $X \in \{-\eta, \dots, \eta\}$, $\mathbb{E}[X] = 0$, and $\text{Var}(X) = \eta$.

We denote by χ_η the discrete distribution on $\mathbb{Z}_q$ obtained by reducing samples from CBD_η modulo q .

Polynomial and vector-valued distributions. We lift χ_η to polynomials in R_q by sampling each coefficient independently:

$$a \leftarrow \chi_\eta^{R_q} \quad \text{means} \quad a(X) = \sum_{i=0}^{n-1} a_i X^i \quad \text{with} \quad a_i \leftarrow \chi_\eta \text{ i.i.d. for } 0 \leq i < n.$$

Similarly, for $k > 1$ we define $\chi_\eta^{R_q^k}$ as the product distribution on R_q^k obtained by sampling each component in R_q according to $\chi_\eta^{R_q}$.

Secret and error distributions. We consider the secret and error distribution, denoted χ_η , where η depends on the target security level (see Section 2.5). For a given parameter set, sampling

$$\mathbf{s}, \mathbf{e} \leftarrow \chi_\eta^{R_q^k}$$

means that each polynomial component and each of its coefficients are chosen independently according to the χ_η distribution. This distribution is chosen so that:

- Secrets and errors are sufficiently small to ensure correctness of the reconciliation mechanism (noise remains within decoding bounds with overwhelming probability).
- At the same time, the induced Module-LWE instances are believed to be hard for current classical and quantum attacks at the targeted security levels, as evidenced by ML-KEM's analysis [15], and is further reaffirmed by the analysis in Section 5.2.

2.1.3 Public Parameters

DKE public parameters. An instance of DKE is specified by a parameter set Π defined as follows:

$$\Pi = (R_q, q, n, k, \chi_\eta, \lambda, \lambda_K, \ell, d, G, H)$$

For the first two levels of security, this may be aligned with one of the ML-KEM parameter sets. For a fixed choice of Π , the following quantities are public and shared by all users:

- The ring

$$R_q = \mathbb{Z}_q[X]/(X^n + 1)$$

with $q = 3329$ ($q = 7681$) and $n = 256$ ($n = 512$).

- The module rank $k \in \{2, 4\}$, determining the dimension of secrets and errors:

$$\mathbf{s}, \mathbf{e} \in R_q^k.$$

- The noise distribution χ_η for secrets and errors, as described in Section 2.1.2.
- A seed length λ (e.g., $\lambda = 256$ bits) for the public matrix seed $\rho \in \{0, 1\}^\lambda$.
- Cryptographic hash and XOF functions, modeled either as concrete primitives (e.g., SM3/KDF-SM3) or as random oracles:
 - $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^*$, an XOF used to generate $\mathbf{A}$ from ρ .
 - $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_K}$, a hash or KDF used to derive the final session key from the public key in DKEM.
- Signal parameter ℓ partitioning $\mathbb{Z}_q$ into 2^ℓ subsets for more fine-grained signals. This may be chosen to match the ML-KEM compression parameter d_v .

- A compression parameter d specifying how many bits are used to represent each coefficient of polynomials. This may be chosen to match the ML-KEM compression parameter d_u .

DKEX parameter sets. Our three-pass authenticated DKEX protocol is directly derived from DKE and a Post-Quantum signature scheme, with the following considerations. During this DKEX specification, the sub-jacent DKE protocol uses secret/error distribution χ_η equal to a centered binomial distribution; the output length of the shared secret and the length of seeds is always equal to n (i.e., $n = \lambda = \lambda_K$), and the hash functions are provisionally provided by ICCS (namely, pseudoHash and pseudoXOF in NGCC). Hence, we only require

$$\pi = (q, n, k, \eta, \ell, d, \text{Sig}),$$

(where Sig is a concrete instantiation of a signature scheme) as a simplified collection of public parameters. In this sense, saying *DKEX is instantiated with a parameter set* $\pi = (q, n, k, \eta, \ell, d, \text{Sig})$ means that the underlying DKE algorithms are defined with respect to

$$\Pi = (R_q, q, n, k, \chi_\eta, n, \ell, d, G, H),$$

where G and H are hash functions with at least n classical security bits (provisionally, pseudoHash and pseudoXOF provided by NGCC).

Concretely, we propose three instantiations for DKEX:

- **DKEX-128:** choosing the public parameter set $\pi_{128} = (3329, 256, 2, 3, 4, 10, \text{ML-DSA-44})$.
- **DKEX-256:** choosing the public parameter set $\pi_{256} = (3329, 256, 4, 2, 5, 11, \text{ML-DSA-87})$.
- **DKEX-512:** choosing the public parameter set $\pi_{512} = (7681, 512, 4, 3, 4, 11, \text{ML-DSA-87})$.

The numbers 128, 256 and 512 refer to the target classical security bits of each instantiation. Notice that the choice of the signature scheme is provisional, and might be changed for selected signatures by the NGCC program. Also notice that we will certainly must change the signature used for DKEX-512, given that in such a case, we can only proof that the 512 security bits are achieved in terms of confidentiality since, at our best of our knowledge, any current suitable signature scheme achieves this number of security bits.

2.1.4 Matrix Generation and Sampling Algorithms

Following ML-KEM, the public matrix $\mathbf{A} \in R_q^{k \times k}$ is generated deterministically from a short seed ρ . This allows compact representation of the global public parameter and ensures that both parties obtain identical matrix entries without transmitting the entire matrix.

At the specification level, matrix generation (Alg. 1) is described through an auxiliary procedure SamplePoly, which is treated as a deterministic black-box sampler parameterized by the seed and the matrix coordinates. This keeps the specification aligned with the implemented behavior while avoiding commitment to a particular low-level XOF interface.

Algorithm 1 GenMatrix($\rho, transposed$)

Input: a seed $\rho \in \{0, 1\}^\lambda$ and a flag $transposed \in \{0, 1\}$

Output: $\mathbf{A}$ if $transposed = 0$, or $\mathbf{A}^\top$ otherwise

```
1: for  $i = 0$  to  $k - 1$  do
2:   for  $j = 0$  to  $k - 1$  do
3:     if  $transposed = 1$  then
4:        $\mathbf{A}_{i,j} \leftarrow \text{SamplePoly}(\rho, i, j)$ 
5:     else
6:        $\mathbf{A}_{i,j} \leftarrow \text{SamplePoly}(\rho, j, i)$ 
7:     end if
8:   end for
9: end for
```

We model $\text{SamplePoly}(\rho, x, y)$ as a deterministic, domain-separated procedure which, on input the public seed ρ and two coordinate values $i, j \in \{0, \dots, k - 1\}$, outputs a polynomial in R_q . For fixed inputs, the output is uniquely determined, and for uniformly random ρ the resulting coefficients are intended to be uniformly distributed over $\mathbb{Z}_q$. The concrete realization of this procedure, including how pseudorandom bytes are expanded and mapped to coefficients, is left to the implementation as long as the resulting behavior is consistent with this specification.

We also require an auxiliary sampling procedure:

Algorithm SampleBinomialPoly $_\eta(\mathbf{b})$: On input a bit string $\mathbf{b} \in \{0, 1\}^{2\eta^n}$, compute a polynomial $\mathbf{s} \in R_q$ with coefficients in $[-\eta, \eta]$. If $\mathbf{b}$ is uniformly random, then the coefficients of $\mathbf{s}$ follow the centered binomial distribution χ_η . Such an algorithm is described in Algorithm 8.

For sampling an element $\mathbf{s} \leftarrow \chi_\eta^{R_q^k}$, we use SampleSecret $_\eta$, given in Algorithm 9.

2.2 Core Functions

The following core functions are used in the ephemeral DKE protocol (Section 2.3), which is evolved to the AKE protocol DKEX in 2.4.

2.2.1 Reconciliation Mechanism

The reconciliation mechanism of the original Ding Key Exchange [5, 6] is adapted here using more fine-grained signals; transmitting ℓ bits per coefficient (2^ℓ signal values) to reduce the failure rate is the general key-consensus technique of [16, 17], specialized in this proposal to the ML-KEM ring. The improved reconciliation mechanism consisting of the following two functions: Signal function (Sig) and Reconciliation function (Rec). In order to define them we partition the domain in the following way:

Partitioning the signal domain. Let ℓ be a positive integer such that $2^{\ell+1} \mid (q-1)$. Define $L = \frac{q-1}{2^\ell}$, then $\frac{L}{2}$ is an integer. The signal domain $\mathbb{Z}_q = \{-\frac{q-1}{2}, \dots, \frac{q-1}{2}\}$ is partitioned into 2^ℓ smaller subdomains:

$$\begin{aligned} I_0 &= \left\{ -\frac{L}{2}, \dots, \frac{L}{2} \right\}, \\ I_1 &= \left\{ \frac{L}{2} + 1, \dots, \frac{3L}{2} \right\}, \\ &\vdots \\ I_{2^{\ell-1}} &= \left\{ \frac{q-1-L}{2} + 1, \dots, \frac{q-1}{2} \right\} \cup \left\{ -\frac{q-1}{2}, \dots, -\frac{q-1-L}{2} - 1 \right\}, \\ &\vdots \\ I_{2^\ell-1} &= \left\{ -\frac{3L}{2}, \dots, -\frac{L}{2} - 1 \right\}. \end{aligned}$$

Signal function Sig. Let $\text{Sig} : \mathbb{Z}_q \times \{0, 1\} \rightarrow \{0, 1, \dots, 2^\ell - 1\}$. For an element $x \in \mathbb{Z}_q$, first sample b randomly from $\{0, 1\}$ with equal probability. If $x - b$ lies within the interval I_w , then define the signal of x as $\text{Sig}(x, b) = w$. The purpose of this random sampling is to eliminate statistical bias in the signal function. The signal function naturally extends to $\text{Sig} : R_q \times \{0, 1\}^n \rightarrow \{0, 1, \dots, 2^\ell - 1\}^n$ by applying it to each coefficient of a polynomial. For simplicity, when we denote $\text{Sig}(x)$, we implicitly assume a random b is already sampled. This function is specified in Algorithm 2.

Algorithm 2 $\text{Sig}(f, \mathbf{b})$ (signal function over R_q)

Input: $f \in R_q$, randomness $\mathbf{b} \in \{0, 1\}^n$;

Output: Signal value $\mathbf{w} \in \{0, 1, 2, \dots, 2^\ell - 1\}^n$.

- 1: Initialize $\mathbf{w} = \vec{0}$.
 - 2: **for** $i = 0, \dots, n-1$ **do**
 - 3: Compute the unique $w \in \{0, 1, 2, \dots, 2^\ell - 1\}$ such that $f[i] - \mathbf{b}[i] \pmod{q} \in I_w$.
 - 4: $\mathbf{w}[i] \leftarrow w$
 - 5: **end for**
 - 6: **return** $\mathbf{w}$.
-

Reconciliation function Rec. Let $\text{Rec} : \mathbb{Z}_q \times \{0, 1, \dots, 2^\ell - 1\} \rightarrow \{0, 1\}$. Given a signal w , an element $y \in \mathbb{Z}_q$ is mapped to 0 or 1 according to the following rule: if the signal is 0, indicating y lies in the interval I_0 , then y modulo 2 is taken directly, i.e., 0 if y is even, and 1 if odd. If the signal is $w > 0$, meaning in the interval $I_w \neq I_0$, then y is first reduced by wL , which effectively rotates the interval I_w back to I_0 , and is then taken modulo 2. Formally, this is expressed as:

$$\text{Rec}(y, w) = y - wL \pmod{q} \pmod{2}.$$

The reconciliation function naturally extends to $\text{Rec} : R_q \times \{0, 1, \dots, 2^\ell - 1\}^n \rightarrow \{0, 1\}^n$ by applying it to each coefficient of a polynomial. This function is specified in Algorithm 3.

The partitioned reconciliation function has the following property: If the difference between elements $x, y \in \mathbb{Z}_q$ is even and less than or equal to $(q/2)(1 - 1/2^\ell) - 2$, then based on the signal of y , the reconciliation function maps x and y to the same value. This fact, formalized in the **DKEM** specification, allows both communicating parties to reconcile and obtain the same value; that is, based on one party's signal, two numbers with the same parity and close values are mapped to the same value. When $\ell = 1$, this is consistent with the original Ding Key

Algorithm 3 $\text{Rec}(f, \mathbf{w})$ (Extractor over R_q)

Input: $f \in R_q, \mathbf{w} \in \{0, 1, \dots, 2^\ell - 1\}^n$.**Output:** $\text{ss} \in \{0, 1\}^n$.

```
1: for  $i = 0, \dots, n-1$  do  
2:    $\text{ss}[i] = f[i] - \mathbf{w}[i]L \pmod{q} \pmod{2}$   
3: end for  
4: return  $\text{ss}$ .
```

Exchange [5, 6]. As ℓ increases, the allowable difference grows from approximately $q/4 - 2$ to close to $q/2 - 2$, meaning the signal and reconciliation functions can tolerate larger errors. The role of partitioning the signal interval is that for the signal $w = \text{Sig}(y)$, the range of $y - wL$ is compressed from the original $\{-q/4, \dots, q/4\}$ to $\{-L/2, \dots, L/2\}$. This enables the reconciliation function $\text{Rec}(x, w) = x - wL = (x - y) + (y - wL)$ to tolerate a larger difference between x and y . Consequently, the sampling range allowed during random sampling is expanded, which greatly reduces the failure probability. But this also comes at the cost that the signal leaks ℓ bits for each coefficient.

We prove that even after we reveal the signal w the extracted bit is still perfectly unbiased.

Lemma 2.1. Let $q > 4$ be an odd integer and ℓ be a positive integer such that $2^{\ell+1} \mid (q-1)$. If x is chosen uniformly random from $\mathbb{Z}_q$, then the reconciliation function $\text{Rec}(x, w)$ defined as above is uniformly random conditioned on the signal $\text{Sig}(x) = w \in \{0, 1, \dots, 2^\ell - 1\}$.

Proof. We leave the details to the **DKEM** specification. □

2.2.2 Number-Theoretic Transform (NTT)

This section provides the concrete algorithms we use to compute the operations described in subsection 2.1.1 related to the NTT. We aim to stay consistent with the formulation of [8, 7, 15].

Algorithm 4 $\text{ntt}(f)$

Input: $f \in \mathbb{Z}_q^n$, and $\zeta \in \mathbb{Z}_q$ of order n **Output:** $\hat{f} \in \mathbb{Z}_q^n$

```
1:  $\hat{f} \leftarrow f$   
2:  $r \leftarrow \log_2(n/2)$   
3:  $i \leftarrow 1$   
4: for  $\text{len} = n/2, n/4, \dots, 4, 2$  do  
5:   for  $\text{start} = 0, 2\text{len}, 4\text{len}, \dots, n - 2\text{len}$  do  
6:      $\zeta \leftarrow \zeta^{\text{bit\_rev}_r(i)} \pmod{q}$   
7:      $i \leftarrow i + 1$   
8:     for  $j = \text{start}, \text{start} + 1, \dots, \text{start} + \text{len} - 1$  do  
9:        $t \leftarrow \zeta \hat{f}[j + \text{len}] \pmod{q}$   
10:       $\hat{f}[j + \text{len}] \leftarrow \hat{f}[j] - t \pmod{q}$   
11:       $\hat{f}[j] \leftarrow \hat{f}[j] + t \pmod{q}$   
12:     end for  
13:   end for  
14: end for  
15: return  $\hat{f}$ 
```

- Algorithm 4 and Algorithm 5 compute the forward and inverse transforms using the classical Cooley–Tukey and Gentleman–Sande butterflies [18, 19], with the bit-reversed twiddle-factor ordering of [20]. We follow the concrete formulation also adopted in [8].

Algorithm 5 $\text{inv_ntt}(\hat{f})$

Input: $\hat{f} \in \mathbb{Z}_q^n$, and $\zeta \in \mathbb{Z}_q$ of order $2n/\iota$ **Output:** $f \in \mathbb{Z}_q^n$

```
1:  $f \leftarrow \hat{f}$ 
2:  $r \leftarrow \log_2(n/2)$ 
3:  $i \leftarrow n/\iota - 1$ 
4: for  $\text{len} = 2, 4, \dots, n/2$  do
5:   for  $\text{start} \leftarrow 0, 2\text{len}, 4\text{len}, \dots, n - 2\text{len}$  do
6:      $\zeta \leftarrow \zeta^{\text{bit\_rev}_r(i)} \bmod q$ 
7:      $i \leftarrow i - 1$ 
8:     for  $j \leftarrow \text{start}, \text{start} + 1, \dots, \text{start} + \text{len} - 1$  do
9:        $u \leftarrow f[j]$ 
10:       $v \leftarrow f[j + \text{len}]$ 
11:       $f[j] \leftarrow u + v \pmod{q}$ 
12:       $f[j + \text{len}] \leftarrow \zeta(v - u) \pmod{q}$ 
13:    end for
14:  end for
15: end for
16:  $c \leftarrow (n/2)^{-1} \pmod{q}$ 
17: return  $c \cdot f$ 
```

▷ component-wise multiplication modulo q

Algorithm 6 $\text{base_mul}(a, b, \Gamma)$

Input: $a = (a_0, a_1)$, $b = (b_0, b_1) \in \mathbb{Z}_q^2$ and $\Gamma \in \mathbb{Z}_q$ **Output:** $r = (r_0, r_1) = (a_0 + a_1X)(b_0 + b_1X) \bmod (X^2 - \Gamma)$

```
1:  $r_0 \leftarrow a_0b_0 + \Gamma a_1b_1 \pmod{q}$ 
2:  $r_1 \leftarrow a_0b_1 + a_1b_0 \pmod{q}$ 
3: return  $(r_0, r_1)$ 
```

Algorithm 7 $\text{ntt_mul}(\hat{a}, \hat{b})$

Input: $\hat{a}, \hat{b} \in \mathbb{Z}_q^n$, and $\zeta \in \mathbb{Z}_q$ of order n **Output:** $\hat{c} \in \mathbb{Z}_q^n$ ▷ We also denote this output as $\hat{a} \circ \hat{b}$

```
1:  $r \leftarrow \log_2(n/2)$ 
2: for  $i \leftarrow 0, 1, \dots, n/2 - 1$  do
3:    $\Gamma \leftarrow \zeta^{2\text{bit\_rev}_r(i)+1} \bmod q$ 
4:    $(\hat{c}[2i], \hat{c}[2i+1]) \leftarrow \text{base\_mul}((\hat{a}[2i], \hat{a}[2i+1]), (\hat{b}[2i], \hat{b}[2i+1]), \Gamma)$ 
5: end for
6: return  $\hat{c}$ 
```

- Algorithm 6 performs the degree-one multiplication in $\mathbb{Z}_q[X]/(X^2 - \Gamma)$, and Algorithm 7 assembles the pointwise product $\hat{a} \circ \hat{b}$ over the $n/2$ blocks. Stopping at quadratic factors rather than splitting $X^n + 1$ completely, which permits the smaller moduli we use, follows [14, 21].
- We write NTT and NTT^{-1} for the transforms of Algorithms 4 and 5 applied to each coordinate of a vector of polynomials, and $\circ$ for the pointwise product of Algorithm 7, extended coordinate-wise to vectors and matrices.

2.2.3 Compression and Decompression Functions

We define the compression function for $x \in \mathbb{Z}_q$ as:

$$\text{Compress}_d(x) = \left\lceil \frac{2^d}{q} \cdot x \right\rceil \bmod 2^d. \quad (1)$$

For $y \in \mathbb{Z}_{2^d}$, the decompression function is:

$$\text{Decompress}_d(y) = \left\lfloor \frac{q}{2^d} \cdot y \right\rfloor. \quad (2)$$

Both functions are extended to R_q and R_q^k coefficient-wise. In these cases they are equally denoted for notation simplicity.

Remark 2.2. Compressing and decompressing the public MLWE samples is not possible in the original Ding Key Exchange [5]. This is because successively compressing and decompressing a value in $\mathbb{Z}_q$ introduces a not necessarily even error: let $r(x) = x - \text{Decompress}_d(\text{Compress}_d(x))$. Supposing that d is sufficiently close to $\lceil \log_2 q \rceil$, $|r(x)|$ will be close to 0. However, it might be an odd number, so compressing must be avoided in [5]. In [6], submitters introduced the so-called Round, Recover and RemoveBias functions to overcome this difficulty. Our approach is slightly different: each time we want to communicate a coefficient $x \in \mathbb{Z}_q$ preserving parity, instead of directly compressing x , we compress $\frac{1}{2}x$ (where $\frac{1}{2}$ denotes the inverse of 2 mod q) to obtain x' , and the receiver recovers $2 \cdot x' \bmod q$. It is easy to check that both properties hold: the error introduced is both small and even as long as d is sufficiently close to $\lceil \log_2 q \rceil$.

2.3 Specification of DKE

In this subsection we give a specification of DKE (Algorithms 8, 9, 10, 11 and 12), a two-pass key exchange protocol between an initiator A and a responder B . At the end of the protocol, both parties compute a shared secret $ss \in \{0, 1\}^n$. We discuss the correctness and reconciliation failure probability for this protocol in 3.3.

Algorithm 8 SampleBinomialPoly $_{\eta}(\mathbf{b})$

Input: $\mathbf{b} \in \{0, 1\}^{2 \cdot \eta \cdot n}$.

Output: $s \in R_q$ with coefficients sampled from $\mathcal{B}_{\eta}(\mathbb{Z}_q)$, as long as $\mathbf{b}$ is uniformly random.

```

1: for  $i = 0, \dots, n - 1$  do
2:    $a \leftarrow \sum_{j=0}^{\eta-1} \mathbf{b}[j + 2 \cdot i \cdot \eta]$ 
3:    $b \leftarrow \sum_{j=0}^{\eta} \mathbf{b}[j + \eta + 2 \cdot i \cdot \eta]$ 
4:    $s[i] \leftarrow a - b$ 
5: end for
   return  $s$ .
```

Algorithm 9 $\text{SampleSecret}_\eta(r, \text{nonce})$

Input: $r \in \{0, 1\}^n$, $\text{nonce} \in \mathbb{N}$ **Output:** $\mathbf{s} \in R_q^k$ with coefficients sampled from $\mathcal{B}_\eta(\mathbb{Z}_q)$.

```
1: for  $j = 0, \dots, k - 1$  do
2:    $\mathbf{b} \leftarrow \text{XOF}((r \parallel \text{nonce}); 2 \cdot \eta \cdot n)$ 
3:    $\mathbf{s}[j] \leftarrow \text{SampleBinomialPoly}_\eta(\mathbf{b})$ 
4:    $++ \text{nonce}$ 
5: end for

return  $\mathbf{s}, \text{nonce}$ 
```

Algorithm 10 $\text{DKE.Initiate}(\text{coins})$

Input: randomness $\text{coins} \in \{0, 1\}^n$.**Output:** public key pk .**Output:** private key sk .

```
1:  $(r, \rho) \leftarrow \text{XOF}(\text{coins}; 2n)$  ▷ expand to two pseudorandom  $n$ -bit seeds
2:  $\bar{\mathbf{A}} \leftarrow \text{GenMatrix}(\rho)$  ▷ generate matrix  $\mathbf{A} \in (\mathbb{Z}_q^n)^{k \times k}$ 
3:  $(\mathbf{s}_A, \text{nonce}) \leftarrow \text{SampleSecret}_\eta(r, 0)$  ▷ generate secret  $\mathbf{s}_A \in (\mathbb{Z}_q^n)^k$ 
4:  $\mathbf{e}_A \leftarrow \text{SampleSecret}_\eta(r, \text{nonce})$  ▷ generate error  $\mathbf{e}_A \in (\mathbb{Z}_q^n)^k$ 
5:  $\bar{\mathbf{s}}_A \leftarrow \text{NTT}(\mathbf{s}_A)$  ▷ run NTT  $k$ -times (once for each coordinate of  $\mathbf{s}_A$ )
6:  $\bar{\mathbf{e}}_A \leftarrow \text{NTT}(\mathbf{e}_A)$  ▷ run NTT  $k$ -times (once for each coordinate of  $\mathbf{e}_A$ )
7:  $\bar{\mathbf{u}}_A \leftarrow \bar{\mathbf{A}} \circ \bar{\mathbf{s}}_A + \bar{\mathbf{e}}_A$  ▷ noisy linear system in NTT domain
8:  $pk \leftarrow \text{ByteEncode}(\bar{\mathbf{u}}_A) \parallel \rho$ 
9:  $sk \leftarrow \text{ByteEncode}(\bar{\mathbf{s}}_A)$ 
10: return  $pk, sk$ 
```

2.4 Specification of DKEX

We construct the authenticated key exchange DKEX by adding signatures to DKE. The protocol is specified in Algorithms 13, 14, 15, 16, 17, 18 and 19. The scheme is presented in a visual way in Figure 1.

Algorithm 13 $\text{DKEX.init}_a()$

*Initiator setup***Output:** initiator verification key $pk_A \in \{0, 1\}^{\text{VKBITS}}$.**Output:** signing key $sk_A \in \{0, 1\}^{\text{SIGKBITS}}$.**Output:** state $st_A = \mathcal{E}$.

```
1:  $\rho \leftarrow \{0, 1\}^{|\rho_{\text{SIG.KeyGen}}|}$ 
2:  $(pk_A, sk_A) \leftarrow \text{SIG.KeyGen}(\rho)$  ▷ long-term signature key pair
3:  $st_A \leftarrow \mathcal{E}$ 
4: return  $(pk_A, sk_A, st_A)$ 
```

Algorithm 11 DKE.Response($pk, coins$)

Input: public key pk .▷ initiator's public key pk **Input:** random $coins \in \{0, 1\}^{2n}$.**Output:** response message c .

▷ responder's compressed data with signal

Output: shared secret $ss \in \{0, 1\}^n$.

```
1:  $(\bar{\mathbf{u}}_A, \rho) \leftarrow pk$ 
2:  $(\mathbf{r}, \mathbf{b}) \leftarrow coins$ 
3:  $\bar{\mathbf{A}}^\top \leftarrow \text{GenMatrix}(\rho, 1)$ 
4:  $\mathbf{s}_B, nonce \leftarrow \text{SampleSecret}_\eta(\mathbf{r}, 0)$ 
5:  $\mathbf{e}_B \leftarrow \text{SampleSecret}_\eta(\mathbf{r}, nonce)$ 
6:  $\bar{\mathbf{s}}_B \leftarrow NTT(\mathbf{s}_B)$ 
7:  $\bar{\mathbf{u}}_B \leftarrow \bar{\mathbf{A}}^\top \circ \bar{\mathbf{s}}_B$ 
8:  $\mathbf{u}_B \leftarrow NTT^{-1}(\bar{\mathbf{u}}_B)$ 
9:  $\mathbf{u}_B \leftarrow \mathbf{u}_B + \mathbf{e}_B$ 
10:  $\mathbf{u}'_B \leftarrow \text{Compress}(\mathbf{u}_B, d_B)$  ▷ compress responder's data
11:  $\bar{k}_B \leftarrow \bar{\mathbf{u}}_A \circ \bar{\mathbf{s}}_B$ 
12:  $k_B \leftarrow NTT^{-1}(\bar{k}_B)$ 
13:  $\mathbf{r} \leftarrow \text{XOF}(\mathbf{r} || nonce; 2 \cdot \eta \cdot n)$ 
14:  $e_B \leftarrow \text{SampleBinomialPoly}(\mathbf{r})$ 
15:  $k_B \leftarrow k_B + e_B$ 
16:  $k_B \leftarrow 2 \cdot k_B$ 
17:  $\mathbf{w} \leftarrow \text{Sig}(k_B, \mathbf{b})$  ▷ generate the signal
18:  $c \leftarrow (\mathbf{u}'_B || \mathbf{w})$  ▷ generate response message from responder's data and signal
19:  $ss \leftarrow \text{Rec}(k_B, \mathbf{w})$  ▷ derive the shared secret by applying error reconciliation
20: return  $(c, ss)$ 
```

Algorithm 12 DKE.DeriveSecret(c, sk)

Input: message c .

▷ responder's compressed data with signal

Input: private key $sk = \bar{\mathbf{s}}_A$.**Output:** shared secret $ss \in \{0, 1\}^n$.

```
1:  $(\mathbf{u}'_B, \mathbf{w}) \leftarrow c$ 
2:  $\mathbf{u}''_B \leftarrow \text{Decompress}(\mathbf{u}'_B, d_B)$ 
3:  $\bar{\mathbf{u}}''_B \leftarrow NTT(\mathbf{u}''_B)$ 
4:  $\bar{k}_A \leftarrow \bar{\mathbf{u}}''_B \circ \bar{\mathbf{s}}_A$ 
5:  $k_A \leftarrow 2 \cdot NTT^{-1}(\bar{k}_A)$ 
6:  $ss \leftarrow \text{Rec}(k_A, \mathbf{w})$  ▷ derive the shared secret by applying error reconciliation
7: return  $ss$ 
```

Algorithm 14 DKEX.init_b()

*Responder setup***Output:** responder long-term verification pk_B .**Output:** responder long-term signing keys $sk_B \in \{0, 1\}^{\text{SIGKBITS}}$.**Output:** state $st_B = \mathcal{E}$.

```
1:  $\rho \leftarrow \{0, 1\}^{|\rho_{\text{SIG.KeyGen}}|}$ 
2:  $(pk_B, sk_B) \leftarrow \text{SIG.KeyGen}(\rho)$  ▷ long-term signature key pair
3:  $st_B \leftarrow \mathcal{E}$ 
4: return  $(pk_B, sk_B, st_B)$ 
```

Algorithm 15 DKEX.generate_pass1_msg_a(pk_A)

Input: initiator long-term verification key $pk_A \in \{0, 1\}^{VKBITS}$.

Output: pass-1 message $m_1 \in \{0, 1\}^{PKBITS}$.

Output: updated state $st_A \in \{0, 1\}^{SKBITS+PKBITS+VKBITS}$.

- 1: $\rho_1 \leftarrow \{0, 1\}^N$
 - 2: $(M_1, sk_e) \leftarrow \text{DKE.Initiate}(\rho_1)$ ▷ ephemeral CPA key pair
 - 3: $st_A \leftarrow (sk_e, M_1, pk_A)$ ▷ M_1, pk_A kept in state for pass 3
 - 4: $m_1 \leftarrow M_1$
 - 5: **return** (m_1, st_A)
-

Algorithm 16 DKEX.generate_pass2_msg_b(m_1, pk_A, pk_B, sk_B)

Input: pass-1 message m_1 .

Input: initiator long-term verification key $pk_A \in \{0, 1\}^{VKBITS}$

▷ pre-distributed.

Input: responder long-term verification key pk_B .

Input: responder long-term signing key sk_B .

Output: pass-2 message m_2 .

Output: updated state $st_B \in \{0, 1\}^{2N}$.

- 1: $M_1 \leftarrow m_1$
 - 2: $\rho_2 \leftarrow \{0, 1\}^{2N}$
 - 3: $(M_2, ss) \leftarrow \text{DKE.Response}(M_1, \rho_2)$
 - 4: $T \leftarrow H(\text{LABEL}_A, \text{LABEL}_B, pk_A, pk_B, M_1, M_2; N)$ ▷ $\text{LABEL}_B = \text{"DKEX-KSIG-}\ell\text{-B"}$, with
 $\ell \in \{128, 256, 512\}$
 - 5: $\sigma_B \leftarrow \text{SIG.Sign}(sk_B, T)$ ▷ $\text{LABEL}_A = \text{"DKEX-KSIG-}\ell\text{-A"}$
 - 6: $st_B \leftarrow (ss, T)$
 - 7: $m_2 \leftarrow (M_2, \sigma_B)$
 - 8: **return** (m_2, st_B)
-

Input: pass-2 message m_2 .**Input:** state st_A as set by DKEX.generate_pass1_msg_a.**Input:** initiator signing key sk_A .**Input:** responder static verification key pk_B

▷ pre-distributed.

Output: pass-3 message $m_3 \in \{0, 1\}^{\text{SIGBITS}}$.**Output:** updated state $st_A \in \{0, 1\}^{2N}$, or $\perp$.

```
1:  $(M_2, \sigma_B) \leftarrow m_2$ 
2:  $(sk_e, M_1, pk_A) \leftarrow st_A$ 
3:  $T \leftarrow H(\text{LABEL}_A, \text{LABEL}_B, pk_A, pk_B, M_1, M_2; N)$ 
4:  $b \leftarrow \text{SIG.Verify}(pk_B, T, \sigma_B)$ 
5: if  $b \neq 1$  then
6:   return  $\perp$ 
7: end if
8:  $ss \leftarrow \text{DKE.DeriveSecret}(M_2, sk_e)$ 
9:  $\sigma_A \leftarrow \text{SIG.Sign}(sk_A, T)$ 
10: erase  $sk_e, M_1, pk_A$  from  $st_A$ 
11:  $st_A \leftarrow (ss, T)$ 
12:  $m_3 \leftarrow \sigma_A$ 
13: return  $(m_3, st_A)$ 
```

▷ kex finished on A's side

Input: last responder message m_b .**Input:** state $st_A \in \{0, 1\}^N$ as set by DKEX.generate_pass3_msg_a.**Output:** shared key $ss \in \{0, 1\}^N$.

```
1:  $ss \leftarrow \text{KDF}(st_A; N)$ 
2: return  $ss$ 
```

Input: last initiator message m_3 ▷ i.e. σ_A .**Input:** state $st_B \in \{0, 1\}^{2N}$ as set by DKEX.generate_pass2_msg_b.**Input:** initiator static verification key pk_A

▷ pre-distributed.

Output: shared key $ss \in \{0, 1\}^N$, or $\perp$.

```
1:  $\sigma_A \leftarrow m_3$ 
2:  $(ss, T) \leftarrow st_B$ 
3:  $b \leftarrow \text{SIG.Verify}(pk_A, T, \sigma_A)$ 
4: if  $b \neq 1$  then
5:   return  $\perp$ 
6: end if
7:  $ss \leftarrow \text{KDF}(st_B; N)$ 
8: return  $ss$ 
```

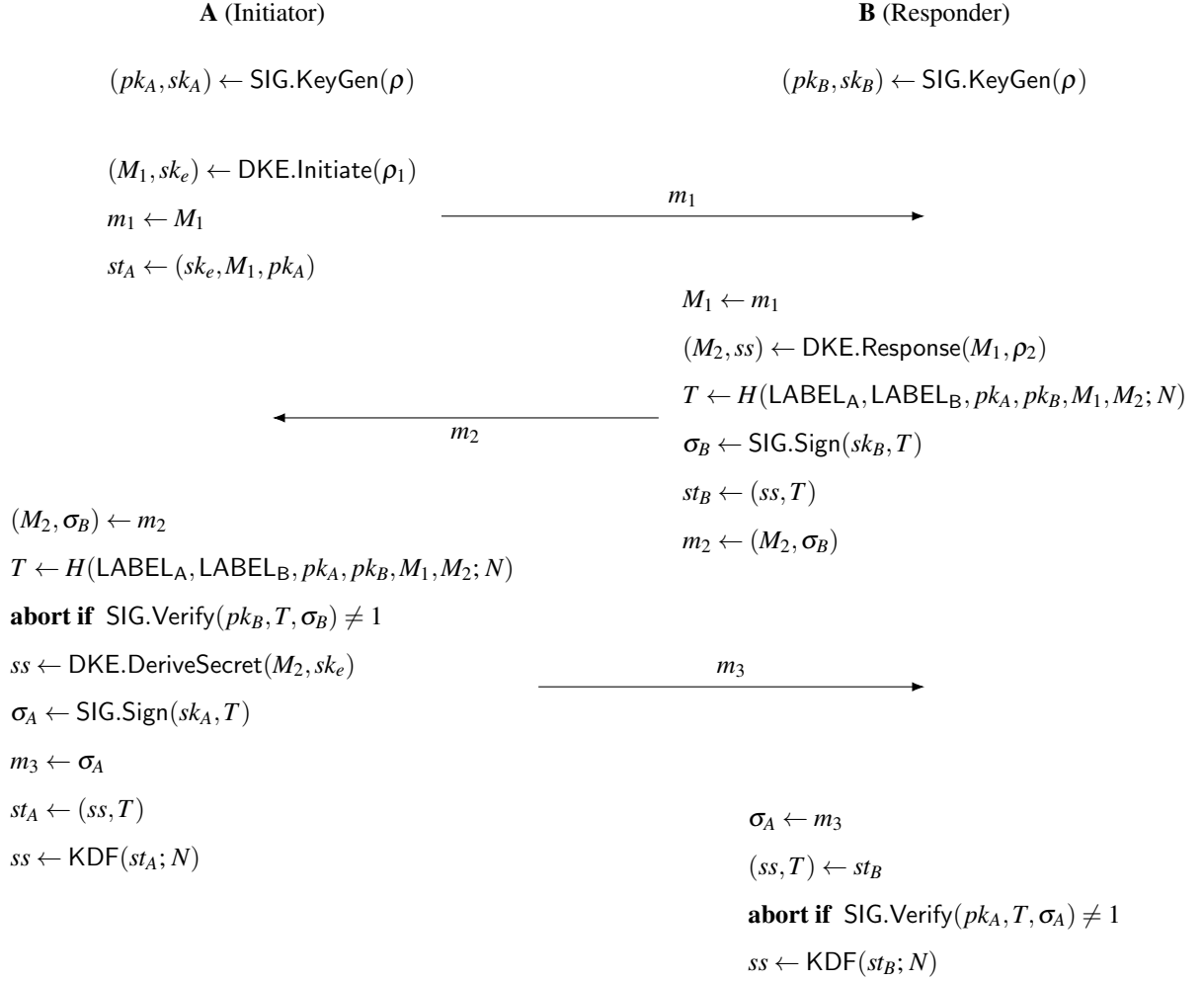


Figure 1: Three-pass mutually authenticated key exchange DKEX based on [10]. The ephemeral key exchange is DKE (CPA); each party additionally holds a long-term signature key pair, with verification keys pk_A, pk_B assumed pre-distributed.

2.5 Parameter Sets

Primitive instantiation of DKEX at each security level is given in Table 1 and parameter sets of DKEX are given in Table 2.

	DKEX-128	DKEX-256	DKEX-512
Ephemeral KEX	DKE-128	DKE-256	DKE-512
Long-term SIG	ML-DSA-44	ML-DSA-87	ML-DSA-87 *
H, KDF	SM3-256	SM3-256	pseudoXOF-512

Table 1: Primitive instantiation of DKEX at each security level.

Name	n	q	k	η	ℓ	d	Failure prob.	NGCC level
DKEX-128	256	3329	2	3	4	10	$2^{-132.7}$	1
DKEX-256	256	3329	4	2	5	11	$2^{-181.2}$	2
DKEX-512	512	7681	4	3	4	11	$2^{-167.0}$	3

Table 2: Summary of parameter sets for DKEX. The failure probability column has been calculated as described in Section 3.3, and using the scripts in [22].

3 Design Rationale

DKEX combines the CPA-secure key exchange DKE with a digital signature, in the signature-authenticated style of Canetti and Krawczyk [10]. The key-exchange layer and its parameters are taken without change from DKE, the same core used by **DKEM** submission. We therefore keep this section short and refer to the **DKEM** specification for the shared material. The four points below are those required by the call [23]: the hard problem, the parameter-selection criteria, the failure probability and its security impact, and other considerations. The justification of the authentication structure itself is given with the security proof.

3.1 The mathematical hard problem

DKEX rests on two assumptions, both over module lattices. Secrecy of the session key comes from the same Module Learning With Errors (MLWE) problem that underlies DKE; we refer to the **DKEM** specification for the statement of MLWE and its role in the security of DKE. Authentication comes from the unforgeability of the signature, which for our instantiation with ML-DSA [9] reduces to MLWE together with Module-SIS. Both layers thus stand on the same lattice family. This is the compatibility the call asks for [23], namely a key exchange and a signature built on one hard problem at a given security level.

3.2 Criteria for Parameter Selection

The key-exchange parameters (q, n, k, η, ℓ, d) are those of DKE-128, DKE-256 and DKE-512, taken unchanged from **DKEM**. The reasons for those, namely the alignment with the ML-KEM ring and modulus at the first two

levels, the single noise parameter η , the added 512-bit set, and the resulting sizes and failure probabilities, are the same as in **DKEM** and are not repeated here.

What is specific to DKEX is the signature. At each level we pick the smallest standardized ML-DSA parameter set whose classical strength reaches the target level: ML-DSA-44 for the 128-bit level and ML-DSA-87 for the 256-bit level (Table 1). For the 512-bit level no standardized post-quantum signature reaches the target. We use ML-DSA-87, the strongest available, which gives NIST category 5, about 256 classical bits [9]. The secrecy of DKEX-512 still meets 512 bits through DKE-512, but its authentication is limited to about 256 bits. As stated in the Feature Statements (Section 6), the target is recovered by substituting a higher-strength signature once one is standardized, with no change to the key-exchange layer.

The hash and key-derivation function H, KDF are built from SM3, as in **DKEM**. At the 128- and 256-bit levels we use SM3-256. At the 512-bit level the shared secret must be at least 512 bits long [23], which is more than one SM3 digest, so we use the SM3-based extendable-output function pseudoXOF-512.

3.3 Failure probability analysis.

DKEX inherits its correctness error from DKE. A run fails only when DKE reconciliation fails, that is, when the two parties recover different raw secrets $s_A \neq s_B$; the transcript-bound keys then differ as well. The failure probability is therefore the DKE reconciliation-failure probability δ , the same quantity computed for **DKEM**, with the same values $2^{-132.7}$, $2^{-181.2}$ and $2^{-167.0}$ at the 128-, 256- and 512-bit levels. We refer to the **DKEM** specification for the derivation.

Impact on security. In the CK/wPFS bound proved in the Security Analysis (Section 5.1), δ appears as a single additive term, so it only has to remain negligible, which it does at every level. Even with the full query budget of the call, 2^{80} activations [23], the expected number of honest sessions that disagree is at most $2^{80}\delta$, which is $2^{-52.7}$ at the 128-bit level and smaller at the others. The failure event here is a disagreement between two honest parties, not a loss of secrecy. Failure-boosting and reaction attacks [24, 25] do not apply: each session uses a fresh, single-use DKE key that is erased afterwards, so an attacker cannot collect failures against a fixed secret [26], and any tampering with the public messages M_1, M_2 is caught by the signed transcript and aborts the session before it completes.

3.4 Other considerations

Why DKE and not DKEM. DKEX uses DKE directly, not the CCA-secure **DKEM**. This is safe here because the DKE keys are ephemeral and used once, and the signed transcript authenticates the exchange. The chosen-ciphertext protection that the Fujisaki–Okamoto transform adds in **DKEM** is meant for static key reuse [26, 27], which DKEX does not do. Authenticating a passively secure key exchange with a signature is the classical signed-Diffie–Hellman design [10], and an ephemeral CPA-secure key exchange of this kind is known to be enough in the post-quantum setting as well [28].

Transcript binding. The final key is $KDF(T, z)$ with $T = H(\text{LABEL}_A, \text{LABEL}_B, pk_A, pk_B, M_1, M_2)$, so it is bound to both identities and both messages. **DKEM** leaves this binding to the protocol built on top of it; DKEX is such a protocol and does the binding here, which also helps prevent unknown-key-share patterns [11, 12].

4 Performance Evaluation

Implementation scope and provenance. DKEX is implemented as a three-pass authenticated key-exchange protocol that combines the DKE arithmetic layer with an ML-DSA/Dilithium signature backend. DKEX-128 uses ML-DSA-44, while DKEX-256 and DKEX-512 use ML-DSA-87. The Reference and Performance Optimization versions expose the same DKEX interface and protocol-message formats.

Reference implementation. The portable implementation executes the complete DKEX exchange using the reference C DKE arithmetic and the vendored reference C ML-DSA/Dilithium signature implementation (see Tables 3, 4, 5).

Parameter set	Avg. time (μ s)	Avg. cycles	Throughput (ops/s)	Static image (B)	Peak RSS (KB)
DKEX-128	1 484.11	3 605 164	673.80	64 250	1 956
DKEX-256	3 245.92	7 801 719	308.08	65 298	1 956
DKEX-512	4 304.16	10 939 769	232.33	66 570	1 956

Table 3: Reference Implementation full-exchange performance. Performance values are arithmetic means.

Parameter set	Signature	Signing pk	Signing sk	Signature size	ss
DKEX-128	ML-DSA-44	1 312	2 560	2 420	32
DKEX-256	ML-DSA-87	2 592	4 896	4 627	32
DKEX-512	ML-DSA-87	2 592	4 896	4 627	64

Table 4: Reference Implementation signature and shared-secret sizes. All sizes are reported in bytes.

Parameter set	State A	State B	Message 1	Message 2	Message 3	Total transmitted
DKEX-128	2 880	1 312	800	3 188	2 420	6 408
DKEX-256	5 696	2 592	1 568	6 195	4 627	12 390
DKEX-512	9 312	2 592	3 392	7 699	4 627	15 718

Table 5: Reference Implementation state and protocol-message sizes. All sizes are reported in bytes.

Benchmarking configuration. Measurements were collected on an Intel Core i9-10885H at 2.40 GHz with 64 GiB of RAM, running Ubuntu 24.04 under Microsoft WSL2 (kernel 6.x). The implementation was compiled with GCC 13.3.0 using `-std=c99`, `-Wpedantic`, `-Wall`, `-Wextra`, `-O2`, `-fcommon`, `-DDKE_FORCE_SCALAR`, `-DDKE_MODE=<128|256|512>`, `-DDKE_HASH=0`, and `-DDKE_RANDOM=0`. Each entry is the arithmetic mean of 128 complete exchanges measured on a single core. Execution time was measured with `clock_gettime(CLOCK_MONOTONIC)`, and CPU cycles were measured with `RDTSC`; throughput is derived from the average execution time. Static image size is `text+data+bss` as reported by `GNU size`, and peak RSS is reported by `getrusage(RUSAGE_SELF)`.

AVX2 Performance Optimization implementation. The AVX2 optimized target preserves the high-level three-pass DKEX protocol and all message formats. It accelerates the DKE polynomial arithmetic through the AVX2 backend, including the linked GNU assembly routines `fq.S`, `ntt.S`, `invntt.S`, `basemul.S`, and `shuffle.S` (see Table 6). The ML-DSA/Dilithium signature backend remains the reference C implementation; consequently, the complete DKEX stack is not described as fully AVX2-optimized. Optimizing the corresponding signature scheme for the target platform would further increase performance. Communication and storage sizes are unchanged from the Reference Implementation.

Parameter set	Avg. time (μ s)	Avg. cycles	Throughput (ops/s)	Static image (B)	Peak RSS (KB)
DKEX-128	1 001.11	2 461 035	998.89	188 607	1 956
DKEX-256	2 179.76	5 705 900	458.77	188 456	1 956
DKEX-512	3 268.28	7 989 899	305.97	181 868	1 956

Table 6: Performance Optimization Version full-exchange results. Performance values are arithmetic means.

Benchmarking configuration. The platform, timing method, sample count, and memory definitions are identical to those used for Table 3. The implementation was compiled with GCC 13.3.0 using `-O3`, `-march=x86-64`, `-mavx2`, `-mtune=native`, `-flto`, `-fomit-frame-pointer`, `-std=c99`, `-Wpedantic`, `-Wall`, `-Wextra`, `-fcommon`, `-DDKE_HASH=0`, and `-DDKE_RANDOM=0`, together with the linked GNU AVX2 assembly sources.

ARMv8-A Performance Optimization implementation. The optimized NEON implementation preserves the high-level three-pass DKEX logic and message formats while replacing the DKE arithmetic backend with NEON-optimized polynomial arithmetic. These links provide hand-optimized routines for NTT/INTT, base multiplication, and polynomial operations. The reference SM3/pseudoXOF is used to generate pseudorandom number in DKEX. Note that this arm64 platform has no stable user-space cycle counter (PMU restricted), so average cycles are not reported; average wall-clock microseconds (μ s) are authoritative and used in Table 7.

Parameter set	Avg. time (μ s)	Throughput (ops/s)	Peak RSS (KB)
DKEX-128	410.73	2 434.69	1 776
DKEX-256	965.74	1 035.48	1 776
DKEX-512	2 040.24	490.14	1 776

Table 7: ARMv8-A Performance Optimization Version full-exchange results. Throughput is derived from the average time.

Benchmarking configuration. Measurements were collected on an ARMv8.2-A architecture, single-core measurement. The implementation was built with the GNU GCC 13 using `-O3`, `-march=armv8.2-a`, `-flto`, `-fomit-frame-pointer`, `-std=c99`, `-Wpedantic`, `-Wall`, `-Wextra`, `-fcommon`, `-DDKE_USE_AARCH64`, `-DDKE_USE_AARCH64_NATIVE`, `-DDKE_HASH=0`, and `-DDKE_RANDOM=0`. The average time entries are arithmetic means of 128 measurements. Each iteration is timed with `clock_gettime(CLOCK_MONOTONIC)`. Peak RSS is process peak resident memory reported by `getrusage(RUSAGE_SELF)`.

ARM Cortex-M4 Performance Optimization implementation. The optimized ARM Cortex-M4 implementation preserves the high-level three-pass DKEX logic and message formats while replacing the DKE arithmetic backend with Cortex-M4-optimized polynomial arithmetic. The linked backend includes the Cortex-M4 assembly routines for NTT/INTT, base multiplication, modular reduction, polynomial operations, matrix accumulation, and SM3 (see Table 8). The ML-DSA/Dilithium signature backend remains the reference C implementation; consequently, the complete DKEX stack is not described as fully Cortex-M4-optimized. Optimizing the corresponding signature scheme for the target platform would further increase performance. Communication and storage sizes are unchanged from the Reference Implementation.

Parameter set	init.a cycles	init.b cycles	pass1 cycles	pass2 cycles	pass3 cycles
DKEX-128	378 454	292 017	112 560	1 296 825	1 059 912
DKEX-256	145 802	138 200	30 605	493 700	521 370
DKEX-512	148 195	152 112	98 605	341 072	471 785

Table 8: Cortex-M4 stage cycle counts from the QEMU self-test harness. These results are additional embedded-platform evidence.

Benchmarking configuration. The Cortex-M4 implementation was built with arm-none-eabi-gcc 14.2.1 using `-mcpu=cortex-m4`, `-mthumb`, `-mfloat-abi=hard`, `-mfpu=fpv4-sp-d16`, `-O2`, `-ffunction-sections`, `-fdata-sections`, `-std=gnu99`, `-nostdlib`, and `-Wl,--gc-sections`. It was executed under QEMU on the netduinoplus2 Cortex-M4 target. Cycle counts were obtained from the 24-bit SysTick down-counter and are the minimum values observed over three complete self-test iterations. The DKE arithmetic uses the Plantard and matrix-accumulation backends, while ML-DSA remains the portable reference C implementation. The Cortex-M4 results are outside the ARMv8.2-A guideline scope and are provided as additional evidence only.

5 Security Statements and Analyses

5.1 Theoretical security

In this section, we first define the underlying security model, namely the Canetti-Krawczyk (CK) security model with weak Perfect Forward Secrecy (wPFS) [10, 13], and prove that the proposed authenticated key exchange DKEX is secure under the CK security model with wPFS.

5.1.1 Security Model

Parties, static keys, and sessions. Let $\mathcal{U}$ be a set of parties. Each party is a probabilistic polynomial-time interactive machine. A party may own a long-term static key pair.

A *session* is one local execution of the protocol at one party. A session has:

- an owner;
- an intended peer;
- a role, initiator I or responder R ;
- a public message transcript;
- a session state st , containing session-specific secret information.

A session is *completed* if its owner outputs a session key.

For DKEX, an initiator session has identifier

$$sid = (\Pi, I, A, B, M_1, M_2),$$

and a responder session has an identifier

$$sid = (\Pi, R, B, A, M_1, M_2),$$

where Π represents the protocol identifier, and (M_1, M_2) are the protocol messages exchanged by parties A and B respectively.

Matching sessions. An initiator session

$$\text{sid} = (\Pi, I, A, B, M_1, M_2)$$

and a responder session

$$\text{sid} = (\Pi, R, B, A, M_1, M_2)$$

are matching if they have opposite roles, opposite owner/peer identities, and the same public protocol messages (M_1, M_2) . We write $\overline{\text{sid}}$ for the matching session of sid , when it exists.

In the proposed DKEX designed with signatures, we provide explicit definitions of adversary queries and freshness.

Adversary queries. The adversary controls message delivery and may issue:

- $\text{Send}(\cdot)$: deliver a message or activation and receive the party's response;
- $\text{SessionKeyReveal}(\text{sid})$: reveal the accepted key of a completed session;
- $\text{SessionStateReveal}(\text{sid})$: reveal the DKE ephemeral state of an incomplete session;
- $\text{Corrupt}(P)$: reveal party P 's long-term signing key;
- $\text{Test}(\text{sid})$: receive either the real session key or a uniform random key.

$\text{SessionStateReveal}()$ does not reveal signing randomness, expanded signing key material, or protected long-term-key subroutine state. As discussed in [29], revealing the random coins used during the computation of any signature in a classical signature scheme can enable an adversary to calculate that party's long-term secret key. Therefore, we refrain from disclosing any randomness in this type of signature scheme.

Definition 5.1 (Freshness for CK/wPFS). Let sid^* be a completed session owned by an honest party A with honest peer B . If a matching session exists, denote it by $\overline{\text{sid}}^*$. The session sid^* is fresh if all of the following hold:

1. The adversary does not ask $\text{SessionKeyReveal}(\text{sid}^*)$.
2. If $\overline{\text{sid}}^*$ exists, the adversary does not ask $\text{SessionKeyReveal}(\overline{\text{sid}}^*)$.
3. The adversary does not ask $\text{SessionStateReveal}(\text{sid}^*)$.
4. If $\overline{\text{sid}}^*$ exists, the adversary does not ask $\text{SessionStateReveal}(\overline{\text{sid}}^*)$.
5. If $\overline{\text{sid}}^*$ exists, the adversary may ask $\text{Corrupt}(A)$ and $\text{Corrupt}(B)$, revealing both long-term signing keys, only after both matching sessions have completed. This is the weak-forward-secrecy case.
6. If no matching session exists, the claimed peer's long-term signing key is not corrupted before sid^* accepts.

Security experiment. The AKE experiment in the CK model proceeds as follows.

- The challenger initializes parties and static keys.
- The adversary adaptively makes send, Reveal, StateReveal, and Corrupt queries.
- At some point, the adversary chooses a completed fresh session sid^* and asks $\text{Test}(\text{sid}^*)$.
- The challenger samples $b \leftarrow \{0, 1\}$. If $b = 0$, it returns the real accepted session key of sid^* . If $b = 1$, it returns a uniformly random string of the same length.
- The adversary continues making allowed queries, preserving freshness, and eventually outputs a guess b' .

The adversary wins if $b' = b$ and sid^* remains fresh.

CK security with weak perfect forward secrecy (CK/wPFS). An AKE protocol is secure in the CK model with weak forward secrecy if the following conditions hold.

- **Correctness.** If two honest parties complete matching sessions, then, except with negligible probability, they output the same session key.
- **Key indistinguishability.** For any PPT adversary $\mathcal{A}$,

$$\text{Adv}_{\Pi}^{\text{CK-wPFS}}(\mathcal{A}) = \left| \Pr[b' = b] - \frac{1}{2} \right|$$

is negligible for every fresh test session sid^* in the following cases:

- **No matching session.** No matching session $\overline{\text{sid}^*}$ exists.
- **wPFS.** A matching session $\overline{\text{sid}^*}$ exists, and the adversary is additionally given the long-term static secret keys of the owners of sid^* and $\overline{\text{sid}^*}$, while the test-session and matching-session states remain unrevealed by freshness.

Definition 5.2 (Derived-key indistinguishability). In the key indistinguishability experiment of DKE, the challenger samples an honest DKE transcripts

$$(M_1, sk_e) \leftarrow \text{DKE.Initiate}(\text{coins}), \quad (M_2, s_B) \leftarrow \text{DKE.Response}(M_1, \text{coins}),$$

$$s_A \leftarrow \text{DKE.DeriveSecret}(M_2, sk_e).$$

Let $ss = s_A = s_B$. If $b = 0$, set $ss_b = ss$, otherwise set $ss \leftarrow \mathcal{K}$. The adversary outputs b' . Define

$$\text{Adv}_{\text{DKE}}^{\text{ind}}(\mathcal{B}) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

DKE satisfies derived-key indistinguishability if the advantage $\text{Adv}_{\text{DKE}}^{\text{ind}}(\mathcal{B})$ is negligible for every efficient $\mathcal{B}$.

Signature Scheme. Let

$$\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$$

be a signature scheme. Each party P has a long-term signing key pair

$$(pk_P, sk_P) \leftarrow \text{KeyGen}(1^\lambda),$$

and the verification key pk_P is certified and uniquely bound to identity P .

Definition 5.3 (EUF-CMA). In the EUF-CMA experiment, the challenger samples $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ and gives pk to adversary $\mathcal{F}$. The adversary may adaptively query a signing oracle on messages $M_1, \dots, M_q$ and receive signatures $\sigma_i \leftarrow \text{Sign}(sk, M_i)$. It wins if it outputs (M^*, σ^*) such that

$$\text{Verify}(pk, M^*, \sigma^*) = 1 \quad \text{and} \quad M^* \notin \{M_1, \dots, M_q\}.$$

Its advantage is denoted $\text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{F})$.

5.1.2 CK security with wPFS

Let p_n be the number of honest parties and let q_s be an upper bound on the number of local sessions activated by the adversary.

Theorem 5.1 (CK/wPFS security). Assume:

- (1) DKE has correctness error at most δ ;
- (2) DKE satisfies derived-key indistinguishability;
- (3) Underlying signature is EUF-CMA secure;

Then, for every CK/wPFS adversary $\mathcal{A}$ against Π , there exist adversaries $\mathcal{B}_{\text{dke}}$, $\mathcal{B}_{\text{sig}}$, and $\mathcal{B}_{\text{hash}}$ such that

$$\begin{aligned} \text{Adv}_{\Pi}^{\text{CK-wPFS}}(\mathcal{A}) &\leq \delta + q_s \text{Adv}_{\text{DKE}}^{\text{ind}}(\mathcal{B}_{\text{dke}}) + p_n \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_{\text{sig}}) \\ &\quad + \text{Adv}_H(\mathcal{B}_{\text{hash}}) + \varepsilon_{\text{sid}}. \end{aligned}$$

Here ε_{sid} is the session-identifier collision probability.

Proof. We first demonstrate the correctness of the system, and then establish key indistinguishability via game-based proofs.

Correctness. In an honest matching execution, the initiator computes

$$s_A \leftarrow \text{DeriveSecret}(M_2, sk_e),$$

and the responder obtains s_B during

$$(M_2, s_B) \leftarrow \text{Response}(M_1).$$

Both parties compute the same public transcript hash

$$T = H(\text{LABEL}_A, \text{LABEL}_B, pk_A, pk_B, M_1, M_2).$$

By DKE correctness,

$$\Pr[s_A \neq s_B] \leq \delta.$$

If $s_A = s_B$, then

$$ss_A = \text{KDF}(T, s_A) = \text{KDF}(T, s_B) = ss_B.$$

Therefore, honest matching sessions disagree with probability at most δ .

Game G_0 : real experiment. Let G_0 be the real CK/wPFS experiment.

Let Succ_i denote the event that $\mathcal{A}$ correctly guesses the hidden test bit in game G_i .

No-matching acceptance. Let no_match be the event that the fresh test session accepts, but no matching honest peer session exists. We show that this implies either a signature forgery, a transcript-hash collision, or a negligible session-identifier failure.

First, suppose an honest initiator A accepts a connection with claimed peer B , but no corresponding responder session at B exists. Then A must have verified a valid signature σ_B under pk_B on

$$T = H(\text{LABEL}_A, \text{LABEL}_B, pk_A, pk_B, M_1, M_2).$$

Let

$$U = (\text{LABEL}_A, \text{LABEL}_B, pk_A, pk_B, M_1, M_2)$$

be the raw transcript-hash input. There are three cases.

1. Honest B never signed the digest T . Since freshness excludes corruption of B before this no-matching acceptance, (T, σ_B) is a new-message forgery under pk_B .
2. Honest B signed the same digest T , but only for a different raw hash input $U' \neq U$. Then $H(U') = H(U)$, so a collision in H has been found.
3. Honest B signed the same digest for the same raw input U . Then B created a responder local session with the same (A, B, M_1, M_2) . Except for a session-identifier collision, this is a matching responder session, contradicting no_match .

Similarly, if an honest responder B accepts with claimed peer A , but no matching initiator session at A exists, then B must have verified a valid signature σ_A under pk_A on the same transcript hash $T = H(\text{LABEL}_A, \text{LABEL}_B, pk_A, pk_B)$. The same three-case analysis gives either an EUF-CMA forgery under A 's signing key, a collision in H , or a negligible identifier failure.

Now construct a single-user EUF-CMA forger $\mathcal{B}_{\text{sig}}$. It guesses the signer's identity P^* among the p_n honest parties, embeds its EUF-CMA challenge verification key as pk_{P^*} , generates all other signing keys honestly, and answers signing requests for P^* using its signing oracle. If NoMatch occurs through a new digest under the guessed signer, $\mathcal{B}_{\text{sig}}$ outputs the accepted digest-signature pair as a forgery. A hash-collision adversary $\mathcal{B}_{\text{hash}}$ outputs the two distinct raw hash inputs when Case 2 occurs. Thus,

$$\Pr[\text{NoMatch}] \leq p_n \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_{\text{sig}}) + \text{Adv}_H(\mathcal{B}_{\text{hash}}) + \epsilon_{\text{sid}}.$$

Game G_1 : abort on no-matching acceptance. Define G_1 as G_0 , except that the challenger aborts if `no_match` occurs for the test session. The only executions in which G_0 and G_1 differ are those where `no_match` occurs, hence,

$$|\Pr[\text{Succ}_0] - \Pr[\text{Succ}_1]| \leq p_n \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_{\text{sig}}) + \text{Adv}_H(\mathcal{B}_{\text{hash}}) + \epsilon_{\text{sid}}.$$

In G_1 , whenever the fresh test session accepts, it has a genuine matching honest peer session except for the failure events already bounded above.

Game G_2 : abort on correctness failure. Define G_2 as G_1 , except that the challenger aborts if the test exchange is an honest matching exchange but the two raw secrets differ:

$$s_A \neq s_B.$$

By DKE correctness,

$$|\Pr[\text{Succ}_1] - \Pr[\text{Succ}_2]| \leq \delta.$$

Conditioned on no abort, write

$$z^* = s_A = s_B$$

for the common raw secret of the test exchange, and define

$$T^* = H(\text{LABEL}_A, \text{LABEL}_B, pk_A, pk_B, M_1^*, M_2^*).$$

The test-session key is then

$$ss^* = \text{KDF}(T^*, z^*).$$

Game G_3 : replace the test derived key by random. Define G_3 as G_2 , except that the real test-session key $\text{KDF}(T^*, z^*)$ is replaced by a uniform key $ss_r \xleftarrow{\$} \{0, 1\}^\kappa$. We show

$$|\Pr[\text{Succ}_2] - \Pr[\text{Succ}_3]| \leq q_s \text{Adv}_{\text{DKE}}^{\text{ind}}(\mathcal{B}_{\text{dke}}).$$

The derived-key distinguisher $\mathcal{B}_{\text{dke}}$ receives (M_1^*, M_2^*) from the challenger. It guesses one of the at most q_s local sessions to be the test session. In that guessed session, it uses M_1^* as the initiator public DKE message and M_2^* as the responder public DKE message. Once the session's identities and certified verification keys are fixed, it computes

$$T^* = H(\text{LABEL}_A, \text{LABEL}_B, pk_A, pk_B, M_1^*, M_2^*)$$

and submits T^* to its derived-key challenger. It receives s_B , which is either $\text{KDF}(T^*, z^*)$ or uniform, and uses s_B as the key returned in the Test query for the guessed session.

All signature keys are generated by $\mathcal{B}_{\text{dke}}$, so it can produce all required protocol signatures honestly. Freshness ensures that, in a winning execution, the adversary does not obtain the ephemeral state of the test session or of its matching session and does not obtain their session keys via `SessionKeyReveal`. Thus, the reduction is not required to reveal the hidden DKE states corresponding to the challenge transcript. In the wPFS case, after the matching sessions complete, the adversary may corrupt A and B and obtain sk_A, sk_B ; the reduction can answer these queries because it generated the signing keys itself. These signing keys are independent of the hidden challenge bit and of the raw DKE secret.

If $s_B = \text{KDF}(T^*, z^*)$, the adversary's view is distributed as in G_2 . If s_B is uniform, the adversary's view is distributed as in G_3 . The reduction's test-session guess is correct with probability at least $1/q_s$. Therefore,

$$|\Pr[\text{Succ}_2] - \Pr[\text{Succ}_3]| \leq q_s \text{Adv}_{\text{DKE}}^{\text{ind}}(\mathcal{B}_{\text{dke}}).$$

Final game and calculation. In G_3 , the key returned in the real branch is already uniform, and the key returned in the random branch is also uniform. Hence,

$$\Pr[\text{Succ}_3] = \frac{1}{2}.$$

By the triangle inequality,

$$\begin{aligned} \text{Adv}_{\Pi}^{\text{CK-wPFS}}(\mathcal{A}) &= \left| \Pr[\text{Succ}_0] - \frac{1}{2} \right| \\ &\leq |\Pr[\text{Succ}_0] - \Pr[\text{Succ}_1]| + |\Pr[\text{Succ}_1] - \Pr[\text{Succ}_2]| \\ &\quad + |\Pr[\text{Succ}_2] - \Pr[\text{Succ}_3]| + \left| \Pr[\text{Succ}_3] - \frac{1}{2} \right| \\ &\leq p_n \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_{\text{sig}}) + \text{Adv}_H(\mathcal{B}_{\text{hash}}) + \epsilon_{\text{sid}} \\ &\quad + \delta + q_s \text{Adv}_{\text{DKE}}^{\text{ind}}(\mathcal{B}_{\text{dke}}). \end{aligned}$$

□

5.2 Analysis with respect to known attacks

5.2.1 Attacks against the underlying MLWE problem

The best known attacks against the underlying MLWE problem in DKEX do not significantly exploit the algebraic structure of module lattices. Several works have investigated whether the additional algebraic structure present in Ring-LWE and Module-LWE instances can be leveraged to improve attacks. More concretely, the works in [30, 31] propose quantum algorithms for recovering short generators of principal ideals in cyclotomic rings, which translates to efficient algorithms for Ideal-SVP in certain settings. Similarly, [32] gave efficient quantum algorithms for the principal ideal problem in arbitrary degree number fields. However, subsequent analysis established that these results do not extend to the module lattice setting in any practically threatening way. In particular, as noted in [15], any attempt to use the reduction in [33] from MLWE to Ring-LWE to attack DKEX would produce an Ring-LWE instance with a substantially larger modulus and error, making the resulting problem harder to attack than the original MLWE instance. Most recently, theoretical work on module-BKZ has shown that exploiting module structure can yield subexponential speedups for certain cyclotomic fields [34, 35]; however, this speedup applies only to non-power-of-two cyclotomic fields, so the ring used in DKEX is explicitly outside the scope of this result. While all these algebraic techniques lead to a modest reduction in the estimated cost of certain attacks, the improvement is limited and does not currently lead to practical attacks against standard parameter sets [36]. We therefore analyze the hardness of the MLWE problem as an LWE problem.

Attacks against the LWE problem. Many algorithms exist for solving the LWE problem [37]. However, most combinatorial and algebraic approaches require significantly more samples than are available in the

present scheme, which provides only $m = (k + 1)n$ LWE samples. In particular, BKW-type attacks typically need thousands of samples to succeed [38] and are therefore effectively ruled out in this limited-sample regime. Similarly, algebraic attacks such as the Arora–Ge linearization method and Gröbner basis techniques require substantially more samples than available here and are therefore infeasible [39]. Consequently, the relevant attacks are lattice-reduction–based methods using BKZ [40, 41], namely the primal and dual attacks (and later hybrid attacks), which form the primary basis for estimating the hardness of our LWE problem.

The BKZ algorithm works by iteratively calling an oracle that solves the SVP in a reduced sublattice of block size b , and uses the result to improve the basis of the parent lattice. The number of calls to the SVP oracle is known to remain polynomial [42]. As a first simple approach we introduce the core-SVP hardness methodology, which ignores this polynomial factor and considers only the cost of a single SVP oracle call in dimension b [43]. The results of this analysis can be seen in Table 9. While this approach provides useful intuition, it is too coarse to yield accurate security estimates in the classical framework [15]. More precise concrete estimates are therefore provided in Section 5.2.2.

Enumeration vs sieving. Two algorithmic approaches are used for the SVP oracle in BKZ: enumeration and lattice sieving. Enumeration algorithms systematically explore a bounded search tree of candidate vectors in the lattice basis in order to find a shortest vector. They require relatively little memory, but have running time that grows super-exponentially in the block size b . They are thus only useful for small block dimensions. Lattice sieving algorithms instead maintain large lists of lattice vectors and iteratively combine them to produce shorter vectors. They have exponential complexity both in memory and time, and outperform enumeration around dimension 70 [44]. Modern sieving techniques based on locality-sensitive hashing and filtering achieve a heuristic classical time complexity of approximately $2^{0.292b+o(b)}$ [45]. This last value is the one we considered for the core-SVP hardness estimations. Quantum algorithms can further accelerate lattice sieving. Applying Grover-type search techniques reduces the complexity to approximately $2^{0.265b+o(b)}$ [46, 47], and more recent work using quantum hypercone locality-sensitive filtering improves this bound to roughly $2^{0.257b+o(b)}$ [48, 49]. For the core-SVP hardness estimations we adopted this latter value, corresponding to a stronger quantum attacker assumption and therefore a more conservative bound on our quantum security.

Primal and dual lattice attacks. Following the core-SVP methodology, the hardness of the LWE instance is estimated by determining the block size b for which BKZ reduction is expected to succeed. Two main lattice attacks are considered in this framework: the *primal attack* and the *dual attack*.

Primal attack. The primal attack embeds the LWE instance into a lattice in which the vector containing the secret and error terms appears as a uniquely short vector. Given an LWE instance $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e})$, one constructs the lattice $\Lambda = \{\mathbf{x} \in \mathbb{Z}^{m+kn+1} : (\mathbf{A} \mid \mathbf{I}_m \mid -\mathbf{b})\mathbf{x} \equiv \mathbf{0} \pmod{q}\}$ which has dimension $d = m + kn + 1$ and volume q^m . This lattice contains the vector $\mathbf{v} = (\mathbf{s}, \mathbf{e}, 1)$ whose norm is approximately $\|\mathbf{v}\| \approx \sigma\sqrt{kn+m}$, where σ denotes the standard deviation of the secret and error coefficients. The number of used samples m is numerically optimized among the choices from 0 to $(k + 1)n$, together with the BKZ block size. The success of BKZ is modeled using the geometric series assumption (GSA), producing a basis whose Gram–Schmidt norms follow approximately $\|\mathbf{b}_i^*\| = \delta^{d-2i-1} \cdot \text{Vol}(\Lambda)^{1/d}$, where $\delta = \left((\pi b)^{1/b} \cdot b/2\pi e\right)^{1/(2(b-1))}$. The short vector $\mathbf{v}$ is expected to be detected by BKZ once its projection onto the subspace spanned by the last b Gram–Schmidt vectors becomes shorter than $\|\mathbf{b}_{d-b}^*\|$, that is, the attack is successful if and only if $\sigma\sqrt{b} \leq \delta^{2b-d-1} \cdot q^{m/d}$ [47, 43, 15]. This corresponds to the unique SVP (uSVP) formulation. Nevertheless, in addition to that, the

primal attack can also be viewed as a bounded distance decoding (BDD) problem. After lattice reduction, the target vector lies close enough to the lattice and can be recovered using enumeration if it is within the decoding radius. In practice, both attack variants are considered, and the minimum resulting cost is used as the primal attack cost estimate, which for our parameters is generally the BDD.

Dual attack. The dual attack aims to distinguish LWE samples from uniform by exploiting a statistical bias, so its goal is to find a short vector $\mathbf{w} = (\mathbf{x}, \mathbf{y})$ of length ℓ in the dual lattice $\Lambda^* = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}^m \times \mathbb{Z}^{kn} : \mathbf{A}^T \mathbf{x} \equiv \mathbf{y} \pmod{q}\}$ (mod) which has dimension $d = m + kn$ and volume q^{kn} . In this case, $z = \mathbf{x}^T \mathbf{b} = \mathbf{y}^T \mathbf{s} + \mathbf{x}^T \mathbf{e} \pmod{q}$ follows a Gaussian distribution of standard deviation $\ell\sigma$ if $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ is an LWE sample and is uniform modulo q otherwise. The maximal statistical distance between these two distributions is bounded by $\varepsilon = 4\exp(-2\pi^2\tau^2)$, where $\tau = \ell\sigma/q$, so a vector of length ℓ yields a distinguishing advantage ε against decision-LWE. Under GSA assumption $\ell \approx \|\mathbf{b}_0\| = \delta^{d-1}q^{kn/d}$, for a vector produced by BKZ. Therefore, obtaining an ε -distinguisher requires running BKZ with block size b such that $-2\pi^2\tau^2 \geq \ln(\varepsilon/4)$. To obtain a constant success probability, the advantage must be amplified by repeating the procedure approximately $1/\varepsilon^2$ times. Since sieving algorithms are expected to output about $2^{0.2075b}$ sufficiently short vectors, the total repetition cost is estimated as $R = \max\left(1, \frac{1}{2^{0.2075b}\varepsilon^2}\right)$. This estimate assumes conservatively for the defender that all vectors returned by the sieving algorithm are as short as the shortest one [43, 15].

Hybrid attacks. Both the primal and the dual attacks have hybrid variants, which combine lattice reduction techniques with an exhaustive search over a subset of the secret. In a hybrid attack, the secret vector is decomposed into two parts, one of which is guessed, while the remaining part is recovered via a reduced lattice problem of lower dimension. This trade-off allows the attacker to balance the combinatorial cost of guessing against the cost of lattice reduction. For the primal setting, hybrid variants do not typically yield a significant improvement over standard primal attacks, since the structure of the corresponding uSVP instance does not benefit as strongly from partial secret guessing. In contrast, hybrid dual attacks can provide a noticeable improvement over pure dual attacks when the secret is small, as is our case. The best known attacks against such schemes are therefore hybrid dual attacks, which optimize over the number of guessed secret components and then apply a dual lattice reduction to the resulting reduced instance [50, 51, 52].

5.2.2 Refined gate-count estimate

The core-SVP hardness methodology is too coarse to produce accurate concrete security estimates in the classical setting. To obtain refined estimates, we rely on the Lattice Estimator of Albrecht et al. [53], which is the most widely accepted tool in the cryptographic community for this purpose.

The estimator models lattice reduction using progressive BKZ, in which the BKZ block size increases gradually throughout the reduction process rather than being fixed from the start. This is generally considered more realistic than fixed-block BKZ analyses. We configure the estimator to use the Gama–Nguyen basis shape simulator (GN11) [54] in place of GSA to model the Gram–Schmidt profile of the reduced basis. Unlike GSA, GN11 is calibrated from experimental BKZ data and captures tail effects in the Gram–Schmidt norms that GSA systematically misses. This provides more accurate predictions of BKZ behavior at the large block sizes relevant to cryptographic security estimates.

For the sieving cost model, we use the MATZOV reduction cost model [55] as implemented in the estimator. The estimator automatically accounts for all BKZ reduction costs, progressive overheads, and dimension

Parameter set	Attack	Lattice dimension d	Mode	BKZ blocksize β	$\log_2(\text{gates})$
DKEX-128	Primal	1014	Classical	414	120.9
			Quantum	414	106.4
	Dual	1024	Classical	424	123.8
			Quantum	424	109.0
	Dual hybrid	1024	Classical	395	115.3
			Quantum	401	103.1
DKEX-256	Primal	1930	Classical	890	259.9
			Quantum	890	228.7
	Dual	1942	Classical	904	264.0
			Quantum	904	232.3
	Dual hybrid	1942	Classical	826	241.2
			Quantum	838	215.4
DKEX-512*	Primal	3686	Classical	1881	549.3
			Quantum	1881	483.4
	Dual	3834	Classical	1890	551.9
			Quantum	1890	485.7
	Dual hybrid	3784	Classical	1732	505.7
			Quantum	1753	450.5

Table 9: Core-SVP hardness of the scheme (using ADPS16 [37] and ChaLoy21 [49] models for classical and quantum estimates respectively in the lattice estimator [53, 22]). Note that this analysis is for the underlying DKE, it is not taken into account that the signature instantiation is done with ML-DSA, which may affect the specific values, specially for DKEX-512, since no standardized signature exists covering that number of security bits.

reductions (dimensions-for-free [56]). No additional multiplicative constants or overhead factors are applied manually, unlike earlier analyses such as in [15].

In the original Kyber round-3 security analysis [15], the primal attack was identified as the dominant attack, with the dual attack providing a secondary reference point at comparable or slightly higher cost. Since then, the landscape of dual attacks has evolved significantly. Guo and Johansson [57] and, independently, the MATZOV team [55] proposed substantial improvements to the dual attack by incorporating a Fast Fourier Transform (FFT) step into the dual-sieve framework, claiming reductions of 5–15 bits in the concrete security of Kyber parameter sets, with potentially similar impact on comparable parameter sets. These results attracted significant attention as they appeared to threaten the security margins of several NIST candidates. However, Ducas and Pulles [58] showed that a key *independence heuristic* underlying these improved dual attacks leads to incorrect predictions in parameter regimes directly relevant to cryptographic applications. Their analysis suggests that the primal attack likely regains the status of the most efficient classical attack under a corrected model. Carrier et al. subsequently proposed a rigorous variant of the MATZOV dual attack in [59], replacing modulus switching with an efficient decoding algorithm based on polar codes over $\mathbb{Z}_q$, and confirmed the claimed improvements under this corrected framework, partially restoring the MATZOV security reductions.

Given this landscape, we adopt the MATZOV reduction cost model [55] as a conservative lower bound on the security of DKEX. We emphasize that the independence heuristic underlying the MATZOV dual attack has been called into question in [58], but the corrected analysis in [59] yields results of comparable magnitude under a rigorous foundation. The primal attack costs are not affected by this controversy at all.

The resulting estimates are summarized in Table 10. They represent a lower bound on the true classical security level: the true cost to an adversary is at least as large as reported, and likely larger under more optimistic models for the defender. As shown, the most cost-efficient attack across all parameter sets is the dual-hybrid

Parameter Set	Attack	Lattice dimension d	BKZ blocksize β	$\log_2(\text{gates})$
DKEX-128	Primal	1009	396	142.1
	Dual	1024	424	149.9
	Dual hybrid	1024	387	139.7
DKEX-256	Primal	1914	870	275.4
	Dual	1954	918	288.5
	Dual hybrid	1954	823	262.3
DKEX-512	Primal	3674	1855	552.9
	Dual	3866	1930	574.0
	Dual hybrid	3836	1736	519.9

Table 10: Refined classical security estimates using the MATZOV reduction cost model [55] and GN11 basis shape simulator [54], as implemented in the LWE Estimator of Albrecht et al. [53, 22]. Note that this analysis is for the underlying DKE, it is not taken into account that the signature instantiation is done with ML-DSA, which may affect the specific values, specially for DKEX-512, since no standardized signature exists covering that number of security bits.

attack. In this case, the proposed parameter sets achieve classical security levels of 139.7, 262.3, and 519.9 bits for DKEX-128, DKEX-256, and DKEX-512 respectively. These margins comfortably exceed the target security levels of 128, 256, and 512 bits [23]. Under less aggressive attack models the true security margins are expected to be higher. Note that these values may be modified by the impact of the used signature.

Regarding lattice reduction costs, the most demanding case (DKEX-128) requires BKZ with block size $\beta = 387$. For comparison, the current record for solving SVP instances as of January 2026 is dimension 210 [60], which remains significantly below the block sizes required to attack any DKEX parameter set.

All reported costs are heuristic and rely on assumptions about lattice reduction behavior, SVP oracle complexity, and hybrid attack efficiency. The resulting estimates differ from earlier analyses such as [15], which did not account for hybrid attacks, relied on less refined cost models, and did not incorporate the developments in dual attack methodology described above.

5.2.3 About quantum security

Estimating the concrete security of lattice-based constructions against quantum adversaries remains an open problem, and no community consensus currently exists on a definitive refined cost model analogous to those available for classical attacks. We therefore adopt a core-SVP-based approach, following standard practice in analyses of schemes such as ML-KEM [15], modeling the cost of the quantum SVP oracle as 2^{cb} for block size b . A natural choice is $c = 0.265$, corresponding to the cost of Grover-accelerated lattice sieving [46, 47]. In this work, however, we adopt the slightly tighter value $c = 0.257$ from Chailloux and Loyer [49], which uses quantum random walks instead of plain Grover search and represents a more conservative (attacker-favorable) security assumption. In this setting, the quantum security bits are 103.1, 215.4 and 450.5 for DKEX-128, DKEX-256 and DKEX-512 respectively (see Table 9). These values significantly exceed the target levels of 80, 128, and 256 quantum security bits [23].

5.2.4 Attacks against symmetric primitives

The symmetric primitives currently used in the reference implementation of DKEX are SM3-based constructions provided by the NGCC auxiliary function library, specifically `sm3hash`, `pseudoXOF`, and `pseudohash`.

The NGCC documentation explicitly states that `pseudoXOF` and `pseudohash` are provided solely for correctness verification in the current evaluation round and do not guarantee security, and that they will be replaced

by standardized primitives in subsequent rounds [23]. The security analysis of DKEX with respect to its symmetric components is therefore conditional: we assume that the final ICCS-standardized replacements will satisfy the properties required by the security proof, namely pseudorandomness for the XOF and PRF components, and collision resistance for the hash components.

Regarding SM3 itself, no attack on the full construction is currently known. Known cryptanalytic results are limited to step-reduced variants and do not threaten the full 64-round function [61, 62]. The security proofs model the hash and XOF functions as random oracles. Should the ICCS replacement primitives be based on a different underlying function, this analysis will need to be updated accordingly.

5.2.5 Attacks exploiting reconciliation failures

In DKEX, the key-exchange layer DKE is used entirely ephemerally: both the initiator’s sample and the responder’s sample are generated fresh for each session and erased once the shared secret has been derived. Attacks that exploit reconciliation failures rely on submitting repeated queries against the same key material and accumulating leakage from near-boundary decoding events across sessions. Since no DKE key material persists beyond a single session, there is no long-term target for such accumulation, and this class of attacks does not apply to DKEX.

5.3 Other security properties

Key Compromise Impersonation. The scheme is resistant to Key Compromise Impersonation (KCI) attacks. In fact, even if the long-term signing key of A , sk_A is compromised, this does not enable an adversary to impersonate B to A . This is due to the fact that a session is accepted by A only after successful verification of the signature σ_B under B ’s public verification key pk_B . Therefore, an adversary that knows sk_A but not sk_B cannot successfully authenticate as B without producing a valid forgery against the underlying signature scheme — in this case instantiated with ML-DSA —. The same argument applies symmetrically: compromise of sk_B does not enable impersonation of A to B .

Reflection attacks. A reflection attack occurs when an adversary reuses messages generated by one participant as messages from the peer, either within the same session or across concurrent sessions, in an attempt to make a party authenticate itself. DKEX resists reflection attacks through peer-specific signature verification, explicit role separation, and structurally distinct protocol messages. On the one hand, the first concern is whether σ_B can be substituted where σ_A is expected, or vice versa. This fails at verification: as long as $pk_A \neq pk_B$ the verification will not succeed for the wrong signature. On the other hand, DKEX further incorporates explicit role identifiers $LABEL_A$ and $LABEL_B$ into the authenticated transcript T . These labels ensure that the transcripts associated with the initiator and responder are distinct, even in the case $pk_A = pk_B$. As a result, the protocol remains resistant to self-reflection attacks.

Side-channel resistance. DKEX is designed for constant-time implementation on all target platforms. The performance-critical, secret-dependent operations (NTT-based polynomial arithmetic, centered-binomial sampling, modular reduction, compression and serialization) are inherited directly from the constant-time, public-domain ML-KEM code bases (PQClean [63], mlkem-native [64]) and from the constant-time Plantard arithmetic of [65], none of which use secret-dependent branches, memory indices, or variable-time divisions. The

components specific to DKEX are implemented with the same discipline in all our implementations. The two-layer structure delegates all long-term-secret-dependent computation to the DeriveSecret step, which simplifies side-channel review and localizes the future addition of masking.

Symmetric-layer side channels. Because the symmetric layer is built entirely on SM3, DKEX can draw directly on the hardware and constant-time SM3 implementations available in the commercial cryptography ecosystem, reducing the side-channel exposure of hashing, key derivation, and pseudorandom expansion on deployed platforms.

6 Feature Statements

This section provides explicit feature statements for DKEX against each of the criteria defined by the NGCC [23]: innovativeness, simplicity, flexibility, compatibility, extensibility, and performance advantages on variant platforms. Each feature is argued from specific verifiable design decisions documented in the preceding sections.

Innovativeness One of the main innovations in DKEX is in its DKE layer. In fact, unlike other MLWE-based schemes, instead of building the shared secret using the LPR framework, where a message is embedded in a noisy ciphertext and recovered via approximate decryption, no message is encrypted in DKE. Concretely, both parties instead compute approximations to the same lattice product from their respective MLWE samples and use a signal function to extract one bit of shared secret per coefficient, adapting the approach in [5, 6] to MLWE.

Moreover, the reconciliation mechanism of [5, 6] is adapted to the module setting and tuned to the ML-KEM ring. Transmitting ℓ bits per coefficient over 2^ℓ signal subintervals to trade a small amount of bandwidth for a markedly lower failure probability is the general *key-consensus* approach studied in [16, 17] (and, for $\ell = 1$, the reconciliation of [5, 6, 43]). The contribution of DKEX here is the instantiation of this mechanism on the standardized ML-KEM parameters, together with a self-contained proof (Lemma 2.1) that the extracted bit is perfectly biased given the signal under our randomized-doubling step.

Simplicity. DKEX uses a single noise parameter η across all secret and error vectors at every security level, in contrast to ML-KEM’s two-parameter design. This unifies the sampling into a single CBD_η instantiation, reduces code footprint and hardware area, and eliminates a class of implementation errors arising from applying the wrong parameter in the wrong context.

Flexibility. DKEX provides three parameter sets covering classical security strengths of at least 128, 256, and 512 bits with corresponding quantum-resistant security of at least 80, 128, and 256 bits, spanning the full range mandated by the Submission Requirements [23]. All three share the same construction, only differing in the choice of the parameters. Note that no standardized signature exists yet to cover the last security level, so the instantiation — which has been done using ML-DSA — does not achieve the 512-bit classical security level. Nevertheless, once a signature of that security level is standardized, it suffices to change the instantiation to achieve the target security.

As requested by NGCC, *algorithms can be implemented securely and efficiently on a wide variety of platforms, supporting various tradeoffs between efficiency and cost.* Concretely, in contrast to more costly Gaussian sampling, DKEX samples secret and noise polynomials using CBD, which can be implemented securely and

efficiently across various platforms [43]. Besides, as demonstrated by existing ML-KEM/ML-DSA optimizations, the underlying polynomial arithmetic in DKEX can likewise be efficiently adapted to a broad range of platforms, from high-performance platforms [66, 67, 68] to resource-constrained devices [65, 69, 70]. Furthermore, DKEX can support various time-memory trade-offs on highly constrained IoT devices [71, 72, 73], providing substantial flexibility for secure and efficient deployment across diverse platforms. All three security levels are supported from a shared codebase with compile-time selection, minimizing deployment and maintenance burden.

Compatibility. The symmetric layer of DKEX (hash function, XOF) is based entirely on SM3, making it natively compatible with the Chinese SM cryptographic suite at the specification level.

Furthermore, being based on the MLWE problem, DKEX shares the same mathematical foundation as the dominant PQC standardized schemes. Moreover, for the 128 and 256-bit parameter sets, DKEX uses the same ring, modulus and rank as ML-KEM-512 and ML-KEM-1024, inheriting also the NTT structure. All this allows existing ML-KEM implementations to contribute, and enables existing research and cryptanalysis exposure to apply to this scheme.

Moreover, the instantiation of DKEX is done using ML-DSA for the signature part. However, it can be deployed in environments that require a specific national or international signature standard without altering the key-exchange layer, just by substituting the chosen signature.

Extensibility. DKE, the CPA core of DKEX, is a complete two-party key exchange protocol, where both parties derive the same n -bit shared secret. Thus, inside this same specification there is a whole complete CPA-secure key exchange protocol that could be used for any case where authentication is not a requirement.

Moreover, by adding a Fujisaki-Okamoto-like transform to the CPA core DKE, a CCA-secure KEM can be achieved. This scheme is submitted as separate submission in this NGCC call: **DKEM**.

On the other hand, a structureless-LWE variant can also be obtained by replacing the module polynomial arithmetic with plain-LWE matrix operations while retaining the signal function, and mod-2 extractor unchanged, providing an assumption-diversity extension consistent with the NGCC emphasis on hard-problem diversity.

DKE is exposed directly as the key-exchange layer, meaning that any future improvement to DKE — updated parameters, new NTT optimizations, revised noise distributions... — propagates to DKEX without protocol-level changes. The modular signature interface similarly allows the authentication layer to be upgraded independently, supporting algorithm agility across standardization cycles.

Performance Advantages on Variant Platforms. The reference implementation executes the complete DKEX protocol using portable C, while the performance optimization version accelerates the DKE polynomial-arithmetic layer using x86-64 AVX2 and ARM Cortex-M4 instructions. The signature component remains implemented using the reference C ML-DSA/Dilithium backend, so the submitted optimized implementation is not presented as a fully AVX2- or ARM-Cortex-M4-optimized protocol stack. Optimizing the corresponding signature scheme in the target platforms would further increase the performance of DKEX.

The complete-exchange measurements reported in the Performance Evaluation section show mean execution-time reductions of approximately 32.5%, 32.8%, and 24.1% for DKEX-128, DKEX-256, and DKEX-512, respectively, without changing the protocol-message or storage sizes. The separation between the protocol, DKE,

and signature layers allows architecture-specific backends to be introduced independently while preserving the protocol logic and interoperability.

For resource-constrained platforms, DKEX also includes a Cortex-M4 implementation using optimized Plantard arithmetic and an assembly-optimized SM3 backend. The additional QEMU evidence covers all three parameter sets and reports stage cycle counts for initialization and all three protocol passes. These implementations provide evidence of instruction-set acceleration on x86-64 and functional execution on a Cortex-M4 target.

References

- [1] Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.
- [2] Oded Regev. On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing (STOC)*, pages 84–93. ACM, 2005.
- [3] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On Ideal Lattices and Learning with Errors over Rings. In *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, 2010.
- [4] Adeline Langlois and Damien Stehlé. Worst-Case to Average-Case Reductions for Module Lattices. *Designs, Codes and Cryptography*, 75:565–599, 2015.
- [5] Jintai Ding, Xiang Xie, and Xiaodong Lin. A Simple Provably Secure Key Exchange Scheme Based on the Learning with Errors Problem. Cryptology ePrint Archive, Paper 2012/688, 2012.
- [6] Jintai Ding, Tsuyoshi Takagi, Xinwei Gao, and Yuntao Wang. Ding Key Exchange. NIST’s call for Post-quantum Cryptography Standardization, 2017.
- [7] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS – Kyber: a CCA-secure module-lattice-based KEM. Cryptology ePrint Archive, Paper 2017/634, 2017.
- [8] National Institute of Standards and Technology. FIPS 203: ML-KEM: Module Lattice-Based Key-Encapsulation Mechanism. <https://doi.org/10.6028/NIST.FIPS.203>, 2024. U.S. Department of Commerce, August 2024.
- [9] National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard. Federal Information Processing Standards Publication 204, 2024.
- [10] Ran Canetti and Hugo Krawczyk. Analysis of Key-Exchange Protocols and Their Use for Building Secure Channels. In Birgit Pfitzmann, editor, *Advances in Cryptology — EUROCRYPT 2001*, pages 453–474, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- [11] Simon Blake-Wilson and Alfred Menezes. Unknown Key-Share Attacks on the Station-to-Station (STS) Protocol. In *Public Key Cryptography*, pages 154–170, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [12] Liqun Chen and Qiang Tang. Bilateral Unknown Key-Share Attacks in Key Agreement Protocols. *J. Univers. Comput. Sci.*, 14:416–440, 2008.
- [13] Atsushi Fujioka, Koutarou Suzuki, Keita Xagawa, and Kazuki Yoneyama. Strongly Secure Authenticated Key Exchange from Factoring, Codes, and Lattices. *Designs, Codes and Cryptography*, 76(3):469–504, September 2015.
- [14] Vadim Lyubashevsky and Gregor Seiler. NTTRU: Truly Fast NTRU Using NTT. Cryptology ePrint Archive, Paper 2019/040, 2019.

- [15] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation (Version 3.02). Submission to the NIST Post-Quantum Cryptography Standardization Project, Round 3, 2021. Updated August 4, 2021.
- [16] Zhengzhong Jin and Yunlei Zhao. Optimal Key Consensus in Presence of Noise. Cryptology ePrint Archive, Paper 2017/1058, 2017.
- [17] Yunlei Zhao, Zhengzhong Jin, Boru Gong, and Guanye Sui. KCL (pka OKCN/AKCN/CNKE): Key Consensus from Lattice. Submission to NIST PQC Standardization Project, Round 1.
- [18] James W. Cooley and John W. Tukey. An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [19] W. Morven Gentleman and Gordon Sande. Fast Fourier Transforms: For Fun and Profit. In *Proceedings of the November 7–10, 1966, Fall Joint Computer Conference (AFIPS '66)*, pages 563–578, 1966.
- [20] Patrick Longa and Michael Naehrig. Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography. In *Cryptology and Network Security (CANS 2016)*, volume 10052 of *Lecture Notes in Computer Science*, pages 124–139. Springer, 2016.
- [21] Gregor Seiler. Faster AVX2 optimized NTT multiplication for Ring-LWE lattice cryptography. Cryptology ePrint Archive, Paper 2018/039, 2018.
- [22] *Basque Centre for Applied Mathematics*. DKE Scripts: Failure-Probability, Communication-Cost and Concrete-Security Analysis Tools for DKEM. GitHub repository, 2026. Accessed 2026-06-24.
- [23] Institute of Commercial Cryptography Standards. Announcement on Launching the Next-generation Commercial Cryptographic Algorithms Program (NGCC), October 2025.
- [24] Jan-Pieter D’Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. On the Impact of Decryption Failures on the Security of LWE/LWR Based Schemes. IACR Cryptology ePrint Archive, Report 2018/1089, 2018.
- [25] Jan-Pieter D’Anvers, Qian Guo, Thomas Johansson, Alexander Nilsson, Frederik Vercauteren, and Ingrid Verbauwhede. Decryption Failure Attacks on IND-CCA Secure Lattice-Based Schemes. In *Public-Key Cryptography – PKC 2019*, volume 11443 of *Lecture Notes in Computer Science*, pages 565–598. Springer, 2019.
- [26] Yue Qin, Chi Cheng, Xiaohan Zhang, Yanbin Pan, Lei Hu, and Jintai Ding. A Systematic Approach and Analysis of Key Mismatch Attacks on Lattice-Based NIST Candidate KEMs. Cryptology ePrint Archive, Paper 2021/123, 2021.
- [27] Scott Fluhrer. Cryptanalysis of ring-LWE based key exchange with key share reuse. Cryptology ePrint Archive, Paper 2016/085, 2016.
- [28] Biming Zhou, Haodong Jiang, and Yunlei Zhao. CPA-secure KEMs are also sufficient for Post-Quantum TLS 1.3. Cryptology ePrint Archive, Paper 2024/1360, 2024.

- [29] Brian LaMacchia, Kristin Lauter, and Anton Mityagin. Stronger security of authenticated key exchange. In Willy Susilo, Joseph K. Liu, and Yi Mu, editors, *Provable Security*, pages 1–16, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.
- [30] Ronald Cramer, Léo Ducas, Chris Peikert, and Oded Regev. Recovering Short Generators of Principal Ideals in Cyclotomic Rings. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 559–585. Springer, 2016.
- [31] Ronald Cramer, Léo Ducas, and Benjamin Wesolowski. Short Stickelberger Class Relations and Application to Ideal-SVP. In *Advances in Cryptology – EUROCRYPT 2017*, volume 10210 of *Lecture Notes in Computer Science*, pages 324–348. Springer, 2017.
- [32] Jean-François Biasse and Fang Song. Efficient Quantum Algorithms for Computing Class Groups and Solving the Principal Ideal Problem in Arbitrary Degree Number Fields. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2016)*, pages 893–902. SIAM, 2016.
- [33] Martin R. Albrecht and Amit Deo. Large Modulus Ring-LWE $\geq$ Module-LWE. Cryptology ePrint Archive, Paper 2017/612, 2017.
- [34] Léo Ducas, Lynn Engelberts, and Paola de Perthuis. Predicting Module-Lattice Reduction. Cryptology ePrint Archive, Paper 2025/1904, 2025.
- [35] Paola de Perthuis and Filip Trenkić. Refined Modelling of the Primal Attack, and Variants Against Module-Learning With Errors. Cryptology ePrint Archive, Paper 2025/2195, 2025.
- [36] Han Wu and Guangwu Xu. Enhancing the Dual Attack against MLWE: Constructing More Short Vectors Using Its Algebraic Structure. Cryptology ePrint Archive, Paper 2022/1661, 2022.
- [37] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of Learning with Errors. Cryptology ePrint Archive, Paper 2015/046, 2015.
- [38] Yu Wei, Lei Bi, Xianhui Lu, and Kunpeng Wang. Memory-Efficient BKW Algorithm for Solving the LWE Problem. Cryptology ePrint Archive, Paper 2025/266, 2025.
- [39] Koen de Boer and Wessel van Woerden. Lattice-based Cryptography: A Survey on the Security of the Lattice-based NIST Finalists. Cryptology ePrint Archive, Paper 2025/304, 2025.
- [40] Claus-Peter Schnorr and Martin Euchner. Lattice Basis Reduction: Improved Practical Algorithms and Solving Subset Sum Problems. *Mathematical Programming*, 66(1–3):181–199, 1994.
- [41] Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better Lattice Security Estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2011.
- [42] Guillaume Hanrot, Xavier Pujol, and Damien Stehlé. Terminating BKZ. Cryptology ePrint Archive, Paper 2011/198, 2011.

- [43] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum Key Exchange — A New Hope. In *Proceedings of the 25th USENIX Security Symposium (USENIX Security 16)*, pages 327–343, Austin, TX, USA, 2016. USENIX Association.
- [44] Martin R. Albrecht, Léo Ducas, Gottfried Herold, Elena Kirshanova, Eamonn W. Postlethwaite, and Marc Stevens. The General Sieve Kernel and New Records in Lattice Reduction. Cryptology ePrint Archive, Paper 2019/089, 2019.
- [45] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. Cryptology ePrint Archive, Paper 2015/1128, 2015.
- [46] Thijs Laarhoven. *Search Problems in Cryptography: From Fingerprinting to Lattice Sieving*. PhD thesis, Eindhoven University of Technology, 2016.
- [47] Martin R. Albrecht, Vlad Gheorghiu, Eamonn W. Postlethwaite, and John M. Schanck. Estimating Quantum Speedups for Lattice Sieves. Cryptology ePrint Archive, Paper 2019/1161, 2019.
- [48] Max Heiser. Improved Quantum Hypercone Locality Sensitive Filtering in Lattice Sieving. Cryptology ePrint Archive, Paper 2021/1295, 2021.
- [49] André Chailloux and J. Loyer. Lattice Sieving via Quantum Random Walks. In Mihir Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021 – 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 63–91. Springer, 2021.
- [50] Thomas Espitau, Antoine Joux, and Natalia Kharchenko. On a Hybrid Approach to Solve Small Secret LWE, 2020.
- [51] Martin R. Albrecht. On dual lattice attacks against small-secret LWE and parameter choices in HELib and SEAL. Cryptology ePrint Archive, Paper 2017/047, 2017.
- [52] Jung Hee Cheon, Minki Hhan, Seungwan Hong, and Yongha Son. A Hybrid of Dual and Meet-in-the-Middle Attack on Sparse and Ternary Secret LWE. *IACR Cryptology ePrint Archive*, 2019:1114, 2019.
- [53] Martin R. Albrecht et al. Lattice-Estimator. <https://github.com/malb/lattice-estimator>, 2015–. Accessed: 2026-03-24.
- [54] Nicolas Gama and Phong Q. Nguyen. Predicting Lattice Reduction. In Nigel Smart, editor, *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 31–51, Berlin, Heidelberg, 2008. Springer.
- [55] MATZOV. Report on the Security of LWE: Improved Dual Lattice Attack. Report, Zenodo, April 2022.
- [56] Léo Ducas. Shortest Vector from Lattice Sieving: A Few Dimensions for Free. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, volume 10820 of *Lecture Notes in Computer Science*, pages 125–145, Cham, 2018. Springer.
- [57] Qian Guo and Thomas Johansson. Faster Dual Lattice Attacks for Solving LWE with Applications to CRYSTALS. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, volume 13093 of *Lecture Notes in Computer Science*, pages 33–62, Cham, 2021. Springer.

- [58] Léo Ducas and Ludo N. Pulles. Does the Dual-Sieve Attack on Learning with Errors Even Work? In *Advances in Cryptology – CRYPTO 2023*, volume 14083 of *Lecture Notes in Computer Science*, pages 37–69, Cham, 2023. Springer.
- [59] Kévin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the Impact of a Variant of MATZOV’s Dual Attack on Kyber. In *Advances in Cryptology – CRYPTO 2025*, volume 16000 of *Lecture Notes in Computer Science*, pages 444–476, Cham, 2025. Springer.
- [60] TU Darmstadt Lattice Challenge. Svp challenge. <https://www.latticechallenge.org/svp-challenge/>, 2026. Accessed: 2026-03-26.
- [61] Florian Mendel, Tomislav Nad, and Martin Schl  ffer. Finding Collisions for Round-Reduced SM3, 2012.
- [62] Jian Zou, Wenling Wu, Shuang Wu, Bozhan Su, and Le Dong. Preimage Attacks on Step-Reduced SM3 Hash Function. In *Information Security and Cryptology – ICISC 2011*, 2012.
- [63] Matthias J. Kannwischer, Peter Schwabe, Douglas Stebila, and Thom Wiggers. Improving Software Quality in Cryptography Standardization Projects. In *IEEE European Symposium on Security and Privacy, EuroS&P 2022 - Workshops, Genoa, Italy, June 6-10, 2022*, pages 19–30, Los Alamitos, CA, USA, 2022. IEEE Computer Society.
- [64] Hanno Becker, Matthias J. Kannwischer, and The mlkem-native project authors. mlkem-native: A secure, fast, and portable C90 implementation of ML-KEM / FIPS 203, 2026. Accessed: 2026-06-23.
- [65] Junhao Huang, Jipeng Zhang, Haosong Zhao, Zhe Liu, Ray C. C. Cheung,   etin Kaya Ko  , and Donglong Chen. Improved Plantard arithmetic for lattice-based cryptography. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(4):614–636, 2022.
- [66] Lipeng Wan, Fangyu Zheng, Guang Fan, Rong Wei, Lili Gao, Yuewu Wang, Jingqiang Lin, and Jiankuo Dong. A novel high-performance implementation of CRYSTALS-Kyber with AI accelerator. In *European Symposium on Research in Computer Security*, pages 514–534. Springer, 2022.
- [67] Xinyi Ji, Jiankuo Dong, Tonggui Deng, Pinchang Zhang, Jiafeng Hua, and Fu Xiao. HI-Kyber: A novel high-performance implementation scheme of Kyber based on GPU. *IEEE Transactions on Parallel and Distributed Systems*, 35(6):877–891, 2024.
- [68] Xuan Yu, Jipeng Zhang, Junhao Huang, Donglong Chen, and Lu Zhou. Multi-way High-Throughput Implementation of Kyber. In *International Conference on Information Security*, pages 41–60. Springer, 2024.
- [69] Jipeng Zhang, Yuxing Yan, Junhao Huang, and   etin Kaya Ko  . Optimized software implementation of Keccak, Kyber, and Dilithium on RV {32, 64} IM {B}{V}. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):632–655, 2025.
- [70] DongHyun Shin, YoungBeom Kim, Ayesha Khalid, Maire O’Neill, and Seog Chung Seo. Optimized Implementations of Keccak, Kyber, and Dilithium on the MSP430 Microcontroller. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2026(2):409–440, 2026.

- [71] Leon Botros, Matthias J Kannwischer, and Peter Schwabe. Memory-efficient high-speed implementation of Kyber on Cortex-M4. In *International Conference on Cryptology in Africa*, pages 209–228. Springer, 2019.
- [72] Denisa O. C. Greconici, Matthias J. Kannwischer, and Amber Sprenkels. Compact Dilithium Implementations on Cortex-M3 and Cortex-M4. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 1–24, 2021.
- [73] Joppe W Bos, Joost Renes, and Amber Sprenkels. Dilithium for Memory Constrained Devices. In *International Conference on Cryptology in Africa*, pages 217–235. Springer, 2022.